Multi-Modal Drone-Based Cell Tower Structural Anomaly Detection Pipeline

Domain: Computer Vision · Structural Health Monitoring · Sensor Fusion
Modalities: 4K RGB Video · Thermal Infrared Imaging · Geospatial Telemetry
Task: Real-time detection and classification of structural anomalies in 5G/LTE cell towers
Framework: YOLOv8 (detection backbone) + late-fusion scoring layer

Overview

Cell tower infrastructure is critical for communication networks. Physical inspection of these towers is dangerous, time-consuming, and expensive when done by human climbers. Drones equipped with multiple sensors offer a safer and faster alternative, but the raw sensor data from a drone flight is only useful if there is an automated system that can parse, fuse, and reason over all incoming streams simultaneously.

This notebook documents a complete end-to-end pipeline:

1. **Dataset preparation** — three independent datasets (RGB, Thermal, Telemetry) are cleaned, validated, and aligned by a shared `mission_id` + `frame_id` key.
2. **Model training** — a YOLOv8 detector is trained independently on the RGB dataset and on the Thermal dataset.
3. **Sensor fusion** — detections from both vision models are fused with live telemetry fields using a weighted scoring formula to produce a single confidence-ranked anomaly record per frame.
4. **Evaluation** — precision, recall, mAP@0.5, and mAP@0.5:0.95 are reported for each modality and for the fused system.
5. **Inference replay** — the full pipeline is replayed on a single video or image file to simulate a real drone inspection run.

Table of Contents

#	Section
7	Fusion Layer — Theory & Implementation
8	Evaluation & Metrics
9	Inference Replay on Video / Image
10	Result Visualisation & Reporting

1 — Environment Setup

```
# ── Core dependencies ──────────────────────────────────────────────
import subprocess, sys

packages = [
    "ultralytics",          # YOLOv8
    "opencv-python",        # Video / image I/O
    "pandas",               # Telemetry CSV handling
    "numpy",
    "matplotlib",
    "seaborn",
    "Pillow",
    "tqdm",
    "pyyaml",
    "torch",                # Underlying DL framework
    "torchvision",
]

for pkg in packages:
    subprocess.run([sys.executable, "-m", "pip", "install", "-q",
pkg], check=False)

print("All packages ready.")
```

```
[notice] A new release of pip is available: 26.1.1 -> 26.1.2
[notice] To update, run: pip install --upgrade pip

[notice] A new release of pip is available: 26.1.1 -> 26.1.2
[notice] To update, run: pip install --upgrade pip

[notice] A new release of pip is available: 26.1.1 -> 26.1.2
[notice] To update, run: pip install --upgrade pip

[notice] A new release of pip is available: 26.1.1 -> 26.1.2
[notice] To update, run: pip install --upgrade pip

[notice] A new release of pip is available: 26.1.1 -> 26.1.2
[notice] To update, run: pip install --upgrade pip
```

```
[notice] A new release of pip is available: 26.1.1 -> 26.1.2
[notice] To update, run: pip install --upgrade pip

[notice] A new release of pip is available: 26.1.1 -> 26.1.2
[notice] To update, run: pip install --upgrade pip

[notice] A new release of pip is available: 26.1.1 -> 26.1.2
[notice] To update, run: pip install --upgrade pip

[notice] A new release of pip is available: 26.1.1 -> 26.1.2
[notice] To update, run: pip install --upgrade pip

[notice] A new release of pip is available: 26.1.1 -> 26.1.2
[notice] To update, run: pip install --upgrade pip

All packages ready.


[notice] A new release of pip is available: 26.1.1 -> 26.1.2
[notice] To update, run: pip install --upgrade pip
```

```python
import os, json, glob, warnings
import cv2
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import seaborn as sns
from pathlib import Path
from tqdm import tqdm
from PIL import Image
import yaml
import torch
from ultralytics import YOLO

warnings.filterwarnings("ignore")

# ── Reproducibility ──────────────────────────────────────────────
SEED = 42
np.random.seed(SEED)
torch.manual_seed(SEED)

# ── Device ──────────────────────────────────────────────────────
DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
print(f"Running on: {DEVICE.upper()}")
print(f"PyTorch version: {torch.__version__}")
```

```
Running on: CUDA
PyTorch version: 2.10.0+rocm7.2.4.git3d3aa833
```

---

2 — Dataset Overview & Validation

2.1 Folder Assumptions

The pipeline expects the following layout relative to this notebook. Adjust `BASE_DIR` if your data lives elsewhere.

```
project_root/
│
├── drone_tower_inspection.ipynb   ← this file
│
├── tower_inspection_dataset/      ← RGB dataset
│   ├── dataset.yaml
│   ├── train/images/  train/labels/
│   ├── val/images/    val/labels/
│   └── test/images/   test/labels/
│
├── cell_tower_thermal/            ← Thermal dataset
│   ├── dataset.yaml
│   ├── train/images/  train/labels/
│   ├── val/images/    val/labels/
│   └── test/images/   test/labels/
│
└── telemetry_dataset/             ← Telemetry dataset
    ├── dataset.yaml
    ├── schema.json
    ├── mission_index.csv
    ├── train/telemetry/  train/annotations/  train/mission_metadata/
    ├── val/  (same)
    └── test/ (same)
```

2.2 Class Definitions

Stream	Class ID	Name	Severity
RGB	0	crack	High
RGB	1	rust	Medium
RGB	2	corrosion	Medium
RGB	3	missing_bolt	High
RGB	4	paint_damage	Low
RGB	5	structural_bend	Critical
Thermal	0	tower_structure	—
Thermal	1	antenna_panel	—

Stream	Class ID	Name	Severity
Thermal	2	cable_harness	—
Thermal	3	hotspot_critical	Critical
Thermal	4	hotspot_moderate	High
Thermal	5	hotspot_minor	Medium
Thermal	6	connector_joint	—
Thermal	7	equipment_cabinet	—
Telemetry	0	hotspot_critical	Critical
Telemetry	1	hotspot_moderate	High
Telemetry	2	hotspot_minor	Medium
Telemetry	3	antenna_misalignment	High
Telemetry	4	cable_fault	Critical
Telemetry	5	connector_failure	Critical
Telemetry	6	corrosion	Medium
Telemetry	7	structural_crack	Critical
Telemetry	8	no_anomaly	—

```
# ── Root paths ──────────────────────────────────────────────────────
BASE_DIR       = Path(".")
RGB_DIR        = BASE_DIR / "tower_inspection_dataset"
THERMAL_DIR    = BASE_DIR / "cell_tower_thermal"
TELEMETRY_DIR  = BASE_DIR / "telemetry_dataset"

# ── Class names ─────────────────────────────────────────────────────
RGB_CLASSES = ["crack", "rust", "corrosion", "missing_bolt",
"paint_damage", "structural_bend"]
THERMAL_CLASSES = ["tower_structure", "antenna_panel",
"cable_harness",
                   "hotspot_critical", "hotspot_moderate",
"hotspot_minor",
                   "connector_joint", "equipment_cabinet"]
TELEM_CLASSES = ["hotspot_critical", "hotspot_moderate",
"hotspot_minor",
                 "antenna_misalignment", "cable_fault",
"connector_failure",
                 "corrosion", "structural_crack", "no_anomaly"]

# ── Severity weights (used in fusion)
```

```
_____________________________________________
SEVERITY_WEIGHT = {
    "crack": 0.85,          "rust": 0.55,         "corrosion": 0.65,
    "missing_bolt": 0.80,   "paint_damage": 0.40, "structural_bend":
0.95,
    "hotspot_critical": 0.95, "hotspot_moderate": 0.75,
"hotspot_minor": 0.50,
    "antenna_misalignment": 0.75, "cable_fault": 0.90,
"connector_failure": 0.90,
    "structural_crack": 0.95, "no_anomaly": 0.0,
}

print("Paths configured.")
for name, path in [("RGB", RGB_DIR), ("Thermal", THERMAL_DIR),
("Telemetry", TELEMETRY_DIR)]:
    status = "✓ found" if path.exists() else "✗ NOT found — check
path"
    print(f"  {name:12s}: {path}  [{status}]")
```

```
Paths configured.
  RGB         : tower_inspection_dataset  [✓ found]
  Thermal     : cell_tower_thermal  [✓ found]
  Telemetry   : telemetry_dataset  [✓ found]
```

```
def count_dataset_files(root: Path, splits=("train", "val", "test")):
    """Count images and labels in each split."""
    records = []
    for split in splits:
        img_dir = root / split / "images"
        lbl_dir = root / split / "labels"
        imgs = len(list(img_dir.glob("*.*"))) if img_dir.exists() else
0
        lbls = len(list(lbl_dir.glob("*.txt"))) if lbl_dir.exists()
else 0
        records.append({"split": split, "images": imgs, "labels":
lbls, "matched": imgs == lbls})
    return pd.DataFrame(records)

print("=" * 55)
print("RGB Dataset")
print("=" * 55)
rgb_counts = count_dataset_files(RGB_DIR)
print(rgb_counts.to_string(index=False))

print()
print("=" * 55)
print("Thermal Dataset")
print("=" * 55)
therm_counts = count_dataset_files(THERMAL_DIR)
print(therm_counts.to_string(index=False))
```

```
print()
print("=" * 55)
print("Telemetry Dataset — missions per split")
print("=" * 55)
for split in ("train", "val", "test"):
    telem_path = TELEMETRY_DIR / split / "telemetry"
    n = len(list(telem_path.glob("*.csv"))) if telem_path.exists()
else 0
    print(f"  {split:8s}: {n} mission CSV files")
```

```
============================================================
RGB Dataset
============================================================
split  images  labels  matched
train      71      70    False
  val      20      20     True
 test      10      10     True

============================================================
Thermal Dataset
============================================================
split  images  labels  matched
train      71      70    False
  val      20      20     True
 test      10      10     True

============================================================
Telemetry Dataset — missions per split
============================================================
  train   : 70 mission CSV files
  val     : 20 mission CSV files
  test    : 10 mission CSV files
```

---

3 — Exploratory Data Analysis

3.1 Class Distribution — RGB Dataset

Understanding how often each defect class appears is essential before training. A heavily imbalanced distribution will bias the model toward frequent classes. Below we parse all label files and count bounding-box instances per class.

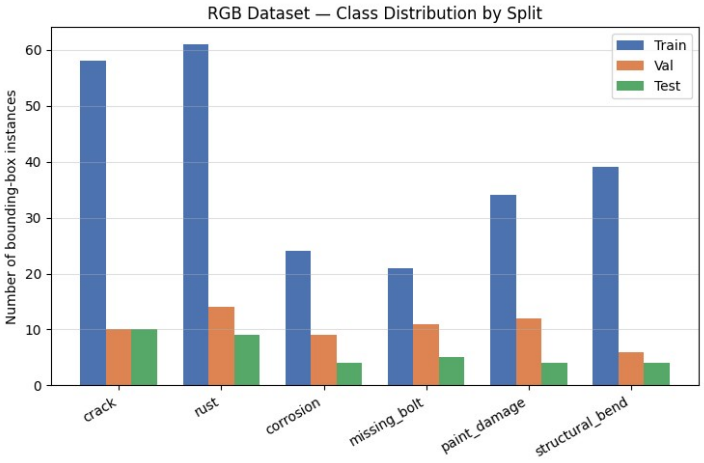
```
def count_class_instances(label_dir: Path, n_classes: int):
    counts = np.zeros(n_classes, dtype=int)
    for lbl_file in label_dir.glob("*.txt"):
        with open(lbl_file) as f:
            for line in f:
```

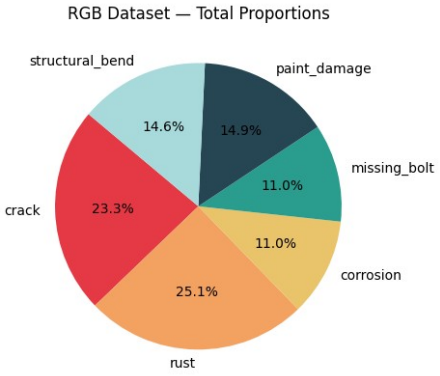
```
                line = line.strip()
                if not line:
                    continue
                cls_id = int(line.split()[0])
                if 0 <= cls_id < n_classes:
                    counts[cls_id] += 1
    return counts

# ── RGB ──────────────────────────────────────────────────────────────
rgb_train_counts = count_class_instances(RGB_DIR / "train" / "labels",
6)
rgb_val_counts   = count_class_instances(RGB_DIR / "val"   / "labels",
6)
rgb_test_counts  = count_class_instances(RGB_DIR / "test"  / "labels",
6)

rgb_df = pd.DataFrame({
    "class": RGB_CLASSES,
    "train": rgb_train_counts,
    "val"  : rgb_val_counts,
    "test" : rgb_test_counts,
})
rgb_df["total"] = rgb_df[["train", "val", "test"]].sum(axis=1)
print("RGB — Bounding-box instance counts per class")
print(rgb_df.to_string(index=False))
```

```
RGB — Bounding-box instance counts per class
          class  train  val  test  total
          crack     58   10    10     78
           rust     61   14     9     84
      corrosion     24    9     4     37
   missing_bolt     21   11     5     37
   paint_damage     34   12     4     50
structural_bend     39    6     4     49
```

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Bar chart — split breakdown
x = np.arange(len(RGB_CLASSES))
w = 0.25
axes[0].bar(x - w, rgb_train_counts, w, label="Train",
color="#4C72B0")
axes[0].bar(x,     rgb_val_counts,   w, label="Val",
color="#DD8452")
axes[0].bar(x + w, rgb_test_counts,  w, label="Test",
color="#55A868")
axes[0].set_xticks(x)
axes[0].set_xticklabels(RGB_CLASSES, rotation=30, ha="right")
```

```
axes[0].set_ylabel("Number of bounding-box instances")
axes[0].set_title("RGB Dataset — Class Distribution by Split")
axes[0].legend()
axes[0].grid(axis="y", alpha=0.4)

# Pie chart — total proportions
palette = ["#e63946", "#f4a261", "#e9c46a", "#2a9d8f", "#264653",
"#a8dadc"]
axes[1].pie(rgb_df["total"], labels=RGB_CLASSES, autopct="%1.1f%%",
            colors=palette, startangle=140)
axes[1].set_title("RGB Dataset — Total Proportions")

plt.tight_layout()
plt.savefig("rgb_class_distribution.png", dpi=150)
plt.show()
```

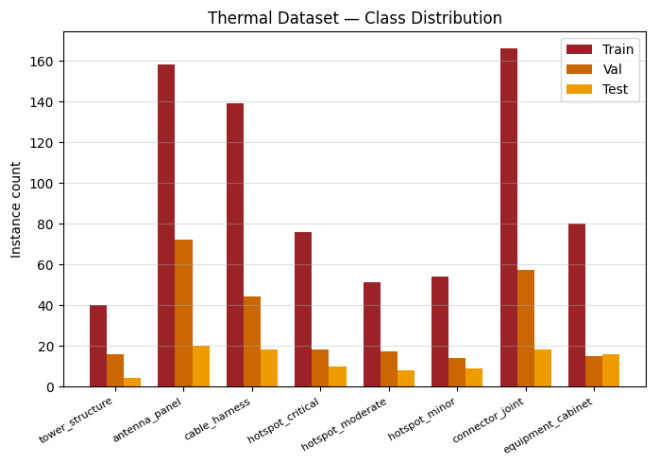


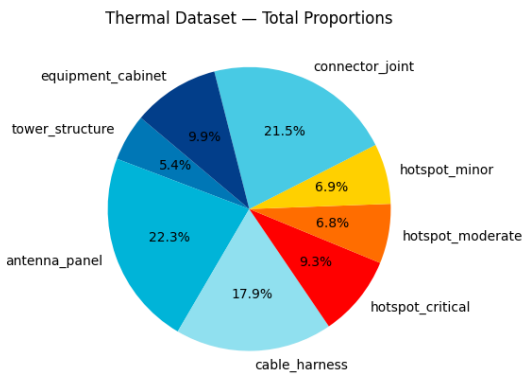



```
# ── Thermal
```

```
th_train = count_class_instances(THERMAL_DIR / "train" / "labels", 8)
th_val   = count_class_instances(THERMAL_DIR / "val"   / "labels", 8)
th_test  = count_class_instances(THERMAL_DIR / "test"  / "labels", 8)

th_df = pd.DataFrame({
    "class": THERMAL_CLASSES,
    "train": th_train,
    "val"  : th_val,
    "test" : th_test,
})
th_df["total"] = th_df[["train", "val", "test"]].sum(axis=1)

fig, axes = plt.subplots(1, 2, figsize=(14, 5))
x = np.arange(len(THERMAL_CLASSES))
axes[0].bar(x - w, th_train, w, label="Train", color="#9b2226")
axes[0].bar(x,     th_val,   w, label="Val",   color="#ca6702")
axes[0].bar(x + w, th_test,  w, label="Test",  color="#ee9b00")
```

```
axes[0].set_xticks(x)
axes[0].set_xticklabels(THERMAL_CLASSES, rotation=30, ha="right",
fontsize=8)
axes[0].set_ylabel("Instance count")
axes[0].set_title("Thermal Dataset — Class Distribution")
axes[0].legend()
axes[0].grid(axis="y", alpha=0.4)

therm_palette = ["#0077b6", "#00b4d8", "#90e0ef", "#ff0000",
"#ff6d00", "#ffd000", "#48cae4", "#023e8a"]
axes[1].pie(th_df["total"], labels=THERMAL_CLASSES, autopct="%1.1f%%",
            colors=therm_palette, startangle=140)
axes[1].set_title("Thermal Dataset — Total Proportions")

plt.tight_layout()
plt.savefig("thermal_class_distribution.png", dpi=150)
plt.show()

print(th_df.to_string(index=False))
```






```
            class  train  val  test  total
  tower_structure     40   16     4     60
    antenna_panel    158   72    20    250
    cable_harness    139   44    18    201
 hotspot_critical     76   18    10    104
 hotspot_moderate     51   17     8     76
    hotspot_minor     54   14     9     77
  connector_joint    166   57    18    241
equipment_cabinet     80   15    16    111
```

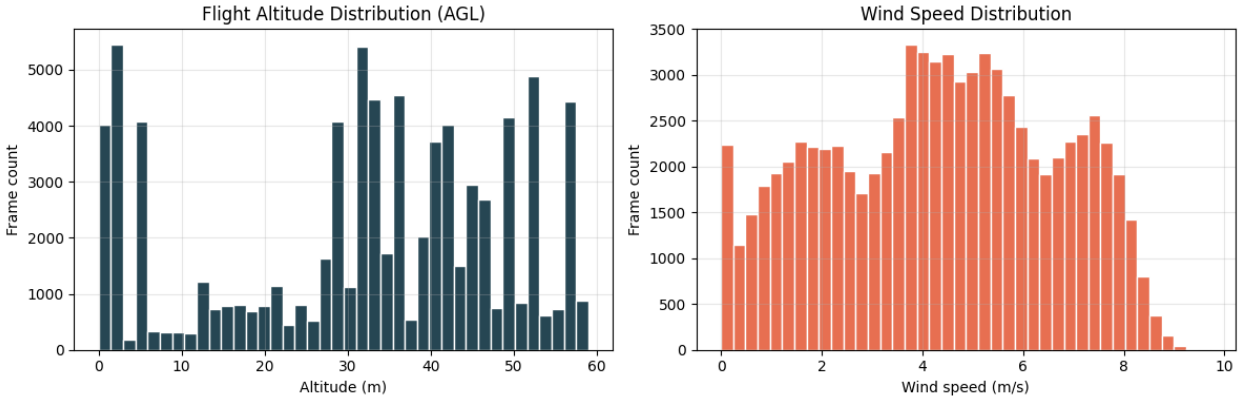
```
# ── Telemetry EDA ──────────────────────────────────────────────────────

# Load all train mission CSVs and summarise key fields

telem_csvs = sorted((TELEMETRY_DIR / "train" /
```

```
"telemetry").glob("*.csv"))

if telem_csvs:
    frames = []
    for fp in tqdm(telem_csvs, desc="Reading telemetry CSVs"):
        df_tmp = pd.read_csv(fp)
        frames.append(df_tmp)
    telem_all = pd.concat(frames, ignore_index=True)

    print(f"Total telemetry frames in train split:
{len(telem_all):,}")
    print(f"Columns ({len(telem_all.columns)}):
{list(telem_all.columns[:10])} ...")

    # Anomaly class frequency from telemetry
    if "class" in telem_all.columns:
        telem_class_counts = telem_all["class"].value_counts()

        fig, ax = plt.subplots(figsize=(10, 4))
        telem_class_counts.plot(kind="bar", ax=ax, color="#457b9d",
edgecolor="white")
        ax.set_title("Telemetry — Anomaly Class Frequency (Train
Split)")
        ax.set_ylabel("Frame count")
        ax.set_xlabel("Anomaly class")
        ax.tick_params(axis="x", rotation=30)
        ax.grid(axis="y", alpha=0.4)
        plt.tight_layout()
        plt.savefig("telemetry_class_freq.png", dpi=150)
        plt.show()
else:
    print("No telemetry CSVs found — skipping EDA. Run after placing
the dataset.")
```

```
Reading telemetry CSVs: 100%|██████████| 70/70 [00:00<00:00,
133.47it/s]

Total telemetry frames in train split: 80,610
Columns (71): ['timestamp_utc', 'mission_id', 'frame_id',
'waypoint_id', 'inspection_phase', 'site_id', 'latitude_deg',
'longitude_deg', 'altitude_m', 'heading_deg'] ...
```

```
# ── Altitude vs wind-speed distribution
# ──────────────────────────────────────
if "telem_all" in dir() and "altitude_m" in telem_all.columns:
    fig, axes = plt.subplots(1, 2, figsize=(12, 4))

    axes[0].hist(telem_all["altitude_m"].dropna(), bins=40,
color="#264653", edgecolor="white")
```

```
    axes[0].set_title("Flight Altitude Distribution (AGL)")
    axes[0].set_xlabel("Altitude (m)")
    axes[0].set_ylabel("Frame count")
    axes[0].grid(alpha=0.3)

    if "wind_speed_ms" in telem_all.columns:
        axes[1].hist(telem_all["wind_speed_ms"].dropna(), bins=40,
color="#e76f51", edgecolor="white")
        axes[1].set_title("Wind Speed Distribution")
        axes[1].set_xlabel("Wind speed (m/s)")
        axes[1].set_ylabel("Frame count")
        axes[1].grid(alpha=0.3)

    plt.tight_layout()
    plt.savefig("telem_flight_stats.png", dpi=150)
    plt.show()
else:
    print("Telemetry data not loaded — skipping altitude/wind plots.")
```



---

4 — Model Architecture

4.1 Why YOLOv8?

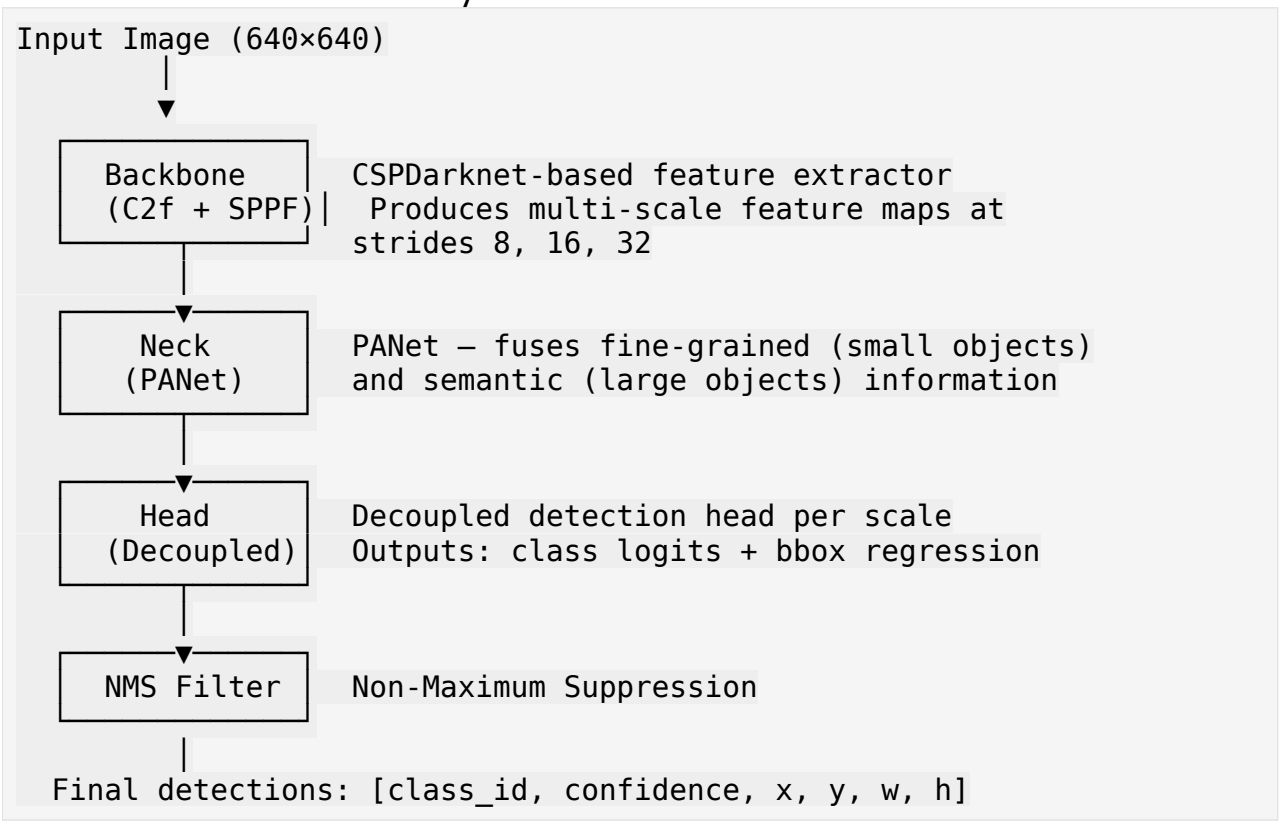
YOLOv8 (You Only Look Once — version 8) is a single-stage object detector. Unlike two-stage detectors (e.g. Faster R-CNN) that first propose regions and then classify them, YOLOv8 performs detection in one forward pass. This makes it significantly faster — a property that matters for real-time drone inspection.

Property	YOLOv8n	YOLOv8s	YOLOv8m
Parameters	3.2 M	11.2 M	25.9 M
GFLOPs	8.7	28.6	78.9
mAP@0.5 (COCO)	37.3	44.9	50.2

Property	YOLOv8n	YOLOv8s	YOLOv8m
Inference (ms)	~1.5	~2.2	~4.1

For this project, **YOLOv8s** (small) is the default. It provides a good balance between speed and accuracy on custom defect datasets. Switch to `yolov8m` if you have a GPU with more than 8 GB VRAM.

4.2 Architecture Summary



4.3 Loss Functions

YOLOv8 uses a composite loss:

$$L_{total} = \lambda_{box} L_{box} + \lambda_{cls} L_{cls} + \lambda_{dfl} L_{dfl}$$

Term	Formula	Purpose
L_{box}	$1 - \text{CIoU}(\hat{b}, b)$	Bounding-box regression
L_{cls}	$\text{BCE}(\hat{p}, p)$	Class probability
L_{dfl}	$-\sum p_i \log \hat{p}_i$	Distribution Focal Loss for precise box edges

CIoU (Complete IoU) is defined as:

$$\mathrm{CIoU} = \mathrm{IoU} - \frac{d^2}{c^2} - \alpha v$$

where:

- d = Euclidean distance between the centres of the predicted and ground-truth boxes
- c = diagonal length of the smallest enclosing box
- $v = \frac{4}{\pi^2}\left(\arctan\frac{w^{gt}}{h^{gt}} - \arctan\frac{w}{h}\right)^2$ — aspect ratio consistency term
- $\alpha = \frac{v}{(1-\mathrm{IoU})+v}$ — trade-off coefficient

CIoU penalises not just overlap (IoU), but also centre distance and aspect-ratio mismatch simultaneously, which produces tighter box predictions compared to plain IoU.

---

5 — RGB Model Training

5.1 Training Configuration

Hyperparameter	Value	Rationale
Model	YOLOv8s	Balance of speed and accuracy
Input size	640 × 640	Matches dataset annotation resolution
Epochs	100	Sufficient for convergence on ~500 images
Batch size	16	Fits in 8 GB GPU
Optimizer	SGD (default)	Robust convergence with cosine LR schedule
Learning rate	0.01 → 0.0001	Cosine annealing
Augmentation	Mosaic, flip, HSV jitter	Improves generalisation on synthetic data
Patience	20	Early stopping
Pretrained	Yes (COCO)	Transfer learning speeds convergence

5.2 Why Transfer Learning?

COCO-pretrained weights already encode low-level visual features (edges, textures, shapes) that are directly useful for structural defect detection. Fine-tuning on our small dataset is far more stable than training from random initialisation.

```
# ── Write dataset YAML for RGB ──────────────────────────────────────
rgb_yaml_content = {
    "path": str(RGB_DIR.resolve()),
    "train": "train/images",
    "val"  : "val/images",
    "test" : "test/images",
    "nc"   : 6,
    "names": RGB_CLASSES,
}

rgb_yaml_path = BASE_DIR / "rgb_dataset.yaml"
with open(rgb_yaml_path, "w") as f:
    yaml.dump(rgb_yaml_content, f, default_flow_style=False)

print(f"RGB dataset YAML written to: {rgb_yaml_path}")
```

```
RGB dataset YAML written to: rgb_dataset.yaml
```

```
# ── Train RGB Model ─────────────────────────────────────────────────
rgb_model = YOLO("yolov8s.pt")   # loads COCO pretrained weights

rgb_results = rgb_model.train(
    data    = str(rgb_yaml_path),
    epochs  = 100,
    imgsz   = 640,
    batch   = 16,
    device  = DEVICE,
    project = "runs/rgb",
    name    = "tower_rgb_v1",
    patience= 20,
    seed    = SEED,
    exist_ok= True,
    verbose = True,
)

RGB_BEST_WEIGHTS = Path("runs/rgb/tower_rgb_v1/weights/best.pt")
print(f"\nBest RGB weights saved at: {RGB_BEST_WEIGHTS}")
```

```
Ultralytics 8.4.69 □ Python-3.12.3 torch-2.10.0+rocm7.2.4.git3d3aa833
CUDA:0 (AMD Instinct MI300X, 196592MiB)
engine/trainer: agnostic_nms=False, amp=True, angle=1.0,
augment=False, auto_augment=randaugment, batch=16, bgr=0.0, box=7.5,
cache=False, cfg=None, classes=None, close_mosaic=10, cls=0.5,
cls_pw=0.0, compile=False, conf=None, copy_paste=0.0,
copy_paste_mode=flip, cos_lr=False, cutmix=0.0, data=rgb_dataset.yaml,
degrees=0.0, deterministic=True, device=0, dfl=1.5, dnn=False,
dropout=0.0, dynamic=False, embed=None, end2end=None, epochs=100,
erasing=0.4, exist_ok=True, fliplr=0.5, flipud=0.0,
```

```
format=torchscript, fraction=1.0, freeze=None, half=False,
hsv_h=0.015, hsv_s=0.7, hsv_v=0.4, imgsz=640, int8=False, iou=0.7,
keras=False, kobj=1.0, line_width=None, lr0=0.01, lrf=0.01,
mask_ratio=4, max_det=300, mixup=0.0, mode=train, model=yolov8s.pt,
momentum=0.937, mosaic=1.0, multi_scale=0.0, name=tower_rgb_v1,
nbs=64, nms=False, opset=None, optimize=False, optimizer=auto,
overlap_mask=True, patience=20, perspective=0.0, plots=True,
pose=12.0, pretrained=True, profile=False, project=runs/rgb,
rect=False, resume=False, retina_masks=False, rle=1.0, save=True,
save_conf=False, save_crop=False,
save_dir=/workspace/shared/Test/Test2/runs/detect/runs/rgb/tower_rgb_v
1, save_frames=False, save_json=False, save_period=-1, save_txt=False,
scale=0.5, seed=42, shear=0.0, show=False, show_boxes=True,
show_conf=True, show_labels=True, simplify=True, single_cls=False,
source=None, split=val, stream_buffer=False, task=detect, time=None,
tracker=botsort.yaml, translate=0.1, val=True, verbose=True,
vid_stride=1, visualize=False, warmup_bias_lr=0.1, warmup_epochs=3.0,
warmup_momentum=0.8, weight_decay=0.0005, workers=8, workspace=None
/assets/Arial.ttf to '/root/.config/Ultralytics/Arial.ttf': 100%
━━━━━━━━━━━━ 755.1KB 15.3MB/s 0.0s
Overriding model.yaml nc=80 with nc=6

                   from  n    params  module
arguments
  0                  -1  1       928  ultralytics.nn.modules.conv.Conv
[3, 32, 3, 2]
  1                  -1  1     18560  ultralytics.nn.modules.conv.Conv
[32, 64, 3, 2]
  2                  -1  1     29056  ultralytics.nn.modules.block.C2f
[64, 64, 1, True]
  3                  -1  1     73984  ultralytics.nn.modules.conv.Conv
[64, 128, 3, 2]
  4                  -1  2    197632  ultralytics.nn.modules.block.C2f
[128, 128, 2, True]
  5                  -1  1    295424  ultralytics.nn.modules.conv.Conv
[128, 256, 3, 2]
  6                  -1  2    788480  ultralytics.nn.modules.block.C2f
[256, 256, 2, True]
  7                  -1  1   1180672  ultralytics.nn.modules.conv.Conv
[256, 512, 3, 2]
  8                  -1  1   1838080  ultralytics.nn.modules.block.C2f
[512, 512, 1, True]
  9                  -1  1    656896
ultralytics.nn.modules.block.SPPF            [512, 512, 5]

 10                  -1  1         0
torch.nn.modules.upsampling.Upsample         [None, 2, 'nearest']

 11             [-1, 6]  1         0
ultralytics.nn.modules.conv.Concat           [1]
```

```
 12                  -1  1    591360  ultralytics.nn.modules.block.C2f
[768, 256, 1]
 13                  -1  1         0
torch.nn.modules.upsampling.Upsample         [None, 2, 'nearest']

 14             [-1, 4]  1         0
ultralytics.nn.modules.conv.Concat           [1]

 15                  -1  1    148224  ultralytics.nn.modules.block.C2f
[384, 128, 1]
 16                  -1  1    147712  ultralytics.nn.modules.conv.Conv
[128, 128, 3, 2]
 17            [-1, 12]  1         0
ultralytics.nn.modules.conv.Concat           [1]

 18                  -1  1    493056  ultralytics.nn.modules.block.C2f
[384, 256, 1]
 19                  -1  1    590336  ultralytics.nn.modules.conv.Conv
[256, 256, 3, 2]
 20             [-1, 9]  1         0
ultralytics.nn.modules.conv.Concat           [1]

 21                  -1  1   1969152  ultralytics.nn.modules.block.C2f
[768, 512, 1]
 22        [15, 18, 21]  1   2118370
ultralytics.nn.modules.head.Detect           [6, 16, None, [128, 256,
512]]
Model summary: 130 layers, 11,137,922 parameters, 11,137,906
gradients, 28.7 GFLOPs

Transferred 349/355 items from pretrained weights
Freezing layer 'model.22.dfl.conv.weight'
AMP: running Automatic Mixed Precision (AMP) checks...
AMP: checks passed □
train: Fast image access □ (ping: 0.1±0.2 ms, read: 1922.9±624.6 MB/s,
size: 792.8 KB)
train: Scanning
/workspace/shared/Test/Test2/tower_inspection_dataset/train/labels.cac
he... 70 images, 0 backgrounds, 0 corrupt: 100% ━━━━━━━━━━━━ 70/70
14.7Mit/s 0.0s
val: Fast image access □ (ping: 0.3±0.3 ms, read: 825.9±462.6 MB/s,
size: 809.5 KB)
val: Scanning
/workspace/shared/Test/Test2/tower_inspection_dataset/val/labels.cache
... 20 images, 0 backgrounds, 0 corrupt: 100% ━━━━━━━━━━━━ 20/20
1.4Mit/s 0.0s
optimizer: 'optimizer=auto' found, ignoring 'lr0=0.01' and
'momentum=0.937' and determining best 'optimizer', 'lr0' and
'momentum' automatically...
```

```
optimizer: AdamW(lr=0.001, momentum=0.9) with parameter groups 57
weight(decay=0.0), 64 weight(decay=0.0005), 63 bias(decay=0.0)
Plotting labels to
/workspace/shared/Test/Test2/runs/detect/runs/rgb/tower_rgb_v1/labels.
jpg...
Image sizes 640 train, 640 val
Using 8 dataloader workers
Logging results to
/workspace/shared/Test/Test2/runs/detect/runs/rgb/tower_rgb_v1
Starting training for 100 epochs...

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 3.1s/it 3.1s
                   all         20         62   0.000805     0.0152
0.00144   0.000864

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 10.6it/s 0.1s
                   all         20         62      0.465     0.0152
0.0166     0.0128

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 10.8it/s 0.1s
                   all         20         62      0.827      0.107
0.105     0.0589

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 7.6it/s 0.1s
                   all         20         62      0.467      0.143
0.174     0.0898

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 7.4it/s 0.1s
                   all         20         62      0.519      0.353
0.388      0.213

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 7.9it/s 0.1s
```

```
                   all         20         62      0.751      0.441
0.428      0.252

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 10.6it/s 0.1s
                   all         20         62      0.704       0.51
0.413      0.237

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 11.5it/s 0.1s
                   all         20         62      0.676      0.568
0.442      0.269

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 13.1it/s 0.1s
                   all         20         62      0.722      0.527
0.453      0.288

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 10.9it/s 0.1s
                   all         20         62      0.673      0.521
0.442      0.286

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 13.0it/s 0.1s
                   all         20         62      0.633      0.514
0.449      0.272

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 13.2it/s 0.1s
                   all         20         62      0.732       0.54
0.468      0.304

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 12.9it/s 0.1s
                   all         20         62       0.71      0.541
```

```
0.448      0.287

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 14.4it/s 0.1s
                   all         20         62      0.824      0.517
0.455        0.3

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 16.9it/s 0.1s
                   all         20         62      0.792      0.568
0.459      0.307

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 15.9it/s 0.1s
                   all         20         62      0.722      0.587
0.459       0.33

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 13.5it/s 0.1s
                   all         20         62      0.696      0.511
0.444      0.309

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 13.7it/s 0.1s
                   all         20         62      0.505      0.592
0.457      0.316

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 14.3it/s 0.1s
                   all         20         62      0.678      0.557
0.434      0.305

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 13.8it/s 0.1s
                   all         20         62      0.603      0.509
0.487      0.327
```

```
      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 14.6it/s 0.1s
                   all         20         62      0.505       0.53
0.466       0.34

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 14.1it/s 0.1s
                   all         20         62      0.707      0.436
0.473      0.328

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 14.7it/s 0.1s
                   all         20         62      0.891      0.422
0.48      0.335

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 14.7it/s 0.1s
                   all         20         62      0.743      0.588
0.481       0.36

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 14.0it/s 0.1s
                   all         20         62      0.782      0.559
0.468      0.346

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 15.2it/s 0.1s
                   all         20         62      0.723      0.541
0.475      0.356

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 14.5it/s 0.1s
                   all         20         62      0.669      0.495
0.505      0.375
```

```
      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 16.7it/s 0.1s
                   all         20         62      0.715      0.551
0.491      0.353

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 16.9it/s 0.1s
                   all         20         62      0.791      0.496
0.468      0.341

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 17.0it/s 0.1s
                   all         20         62      0.794      0.479
0.453      0.338

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 17.7it/s 0.1s
                   all         20         62      0.828      0.496
0.452      0.331

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 15.2it/s 0.1s
                   all         20         62      0.729      0.573
0.458      0.332

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 14.3it/s 0.1s
                   all         20         62      0.677      0.559
0.426      0.323

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 16.3it/s 0.1s
                   all         20         62      0.701      0.483
0.436       0.34

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
```

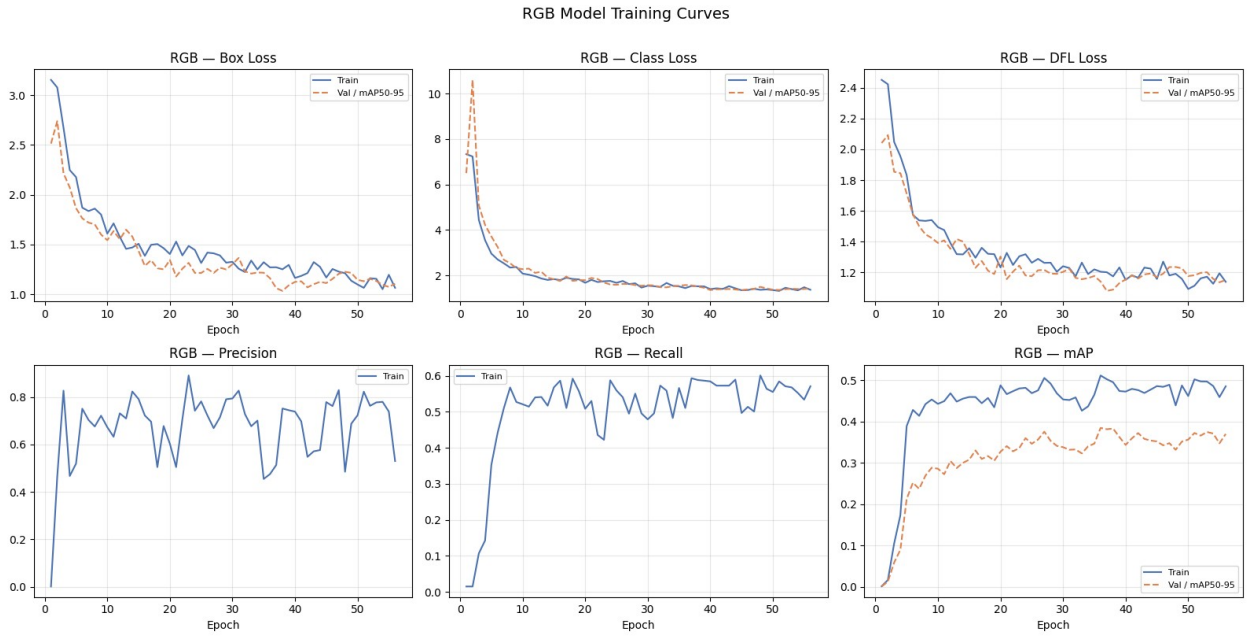
```
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 15.4it/s 0.1s
                   all         20         62      0.455      0.566
0.465      0.346

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 17.0it/s 0.1s
                   all         20         62      0.475      0.511
0.511      0.384

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 17.6it/s 0.1s
                   all         20         62      0.513      0.593
0.502      0.381

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 14.5it/s 0.1s
                   all         20         62      0.752      0.588
0.495      0.383

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 17.7it/s 0.1s
                   all         20         62      0.745      0.587
0.473      0.362

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 16.6it/s 0.1s
                   all         20         62      0.739      0.585
0.472      0.343

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 18.6it/s 0.1s
                   all         20         62      0.698      0.573
0.479      0.359

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
```

```
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 15.4it/s 0.1s
                   all         20         62      0.548      0.573
0.476      0.372

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 15.2it/s 0.1s
                   all         20         62      0.571      0.573
0.469      0.358

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 19.6it/s 0.1s
                   all         20         62      0.577      0.589
0.477      0.354

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 19.2it/s 0.1s
                   all         20         62      0.779      0.497
0.485      0.352

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 16.8it/s 0.1s
                   all         20         62      0.762      0.514
0.484      0.342

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 18.2it/s 0.1s
                   all         20         62       0.83      0.501
0.489      0.347

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 18.7it/s 0.1s
                   all         20         62      0.485      0.601
0.439      0.331

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
```

```
━━━━━━━━━━━━ 1/1 19.2it/s 0.1s
                   all         20         62      0.688      0.564
0.487      0.351

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 17.3it/s 0.1s
                   all         20         62      0.723      0.555
0.461      0.356

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 19.1it/s 0.1s
                   all         20         62      0.823      0.584
0.502      0.372

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 18.8it/s 0.1s
                   all         20         62      0.763      0.571
0.497      0.366

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 17.8it/s 0.1s
                   all         20         62      0.778      0.568
0.496      0.374

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 19.1it/s 0.1s
                   all         20         62       0.78      0.552
0.485      0.371

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 17.6it/s 0.1s
                   all         20         62      0.739      0.534
0.459      0.347

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 18.8it/s 0.1s
```

```
                   all         20         62       0.53      0.571
0.485       0.37
EarlyStopping: Training stopped early as no improvement observed in
last 20 epochs. Best results observed at epoch 36, best model saved as
best.pt.
To update EarlyStopping(patience=20) pass a new patience value, i.e.
`patience=300` or use `patience=0` to disable EarlyStopping.

56 epochs completed in 0.031 hours.
Optimizer stripped from
/workspace/shared/Test/Test2/runs/detect/runs/rgb/tower_rgb_v1/weights
/last.pt, 22.5MB
Optimizer stripped from
/workspace/shared/Test/Test2/runs/detect/runs/rgb/tower_rgb_v1/weights
/best.pt, 22.5MB

Validating
/workspace/shared/Test/Test2/runs/detect/runs/rgb/tower_rgb_v1/weights
/best.pt...
Ultralytics 8.4.69 □ Python-3.12.3 torch-2.10.0+rocm7.2.4.git3d3aa833
CUDA:0 (AMD Instinct MI300X, 196592MiB)
Model summary (fused): 73 layers, 11,127,906 parameters, 0 gradients,
28.4 GFLOPs
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 15.9it/s 0.1s
                   all         20         62      0.474      0.511
0.511      0.384
                 crack          9         10       0.38        0.4
0.399      0.317
                  rust         12         14      0.588          1
0.775      0.638
            corrosion          9          9      0.344      0.444
0.655      0.521
          missing_bolt         10         11      0.899      0.636
0.635       0.53
          paint_damage         10         12      0.635      0.583
0.599      0.298
       structural_bend          6          6          0          0
0.00307   0.000489
Speed: 0.0ms preprocess, 0.5ms inference, 0.0ms loss, 0.8ms
postprocess per image
Results saved to
/workspace/shared/Test/Test2/runs/detect/runs/rgb/tower_rgb_v1

Best RGB weights saved at: runs/rgb/tower_rgb_v1/weights/best.pt

# ── Training curves ──────────────────────────────────────────────────
results_csv = Path("runs/detect/runs/rgb/tower_rgb_v1/results.csv")
```

```
if results_csv.exists():
    res_df = pd.read_csv(results_csv)
    res_df.columns = res_df.columns.str.strip()

    fig, axes = plt.subplots(2, 3, figsize=(16, 8))
    plot_pairs = [
        ("train/box_loss",   "val/box_loss",   "Box Loss",
axes[0, 0]),
        ("train/cls_loss",   "val/cls_loss",   "Class Loss",
axes[0, 1]),
        ("train/dfl_loss",   "val/dfl_loss",   "DFL Loss",
axes[0, 2]),
        ("metrics/precision(B)", None,          "Precision",
axes[1, 0]),
        ("metrics/recall(B)",    None,          "Recall",
axes[1, 1]),
        ("metrics/mAP50(B)",     "metrics/mAP50-95(B)", "mAP",
axes[1, 2]),
    ]

    for train_col, val_col, title, ax in plot_pairs:
        if train_col in res_df.columns:
            ax.plot(res_df["epoch"], res_df[train_col], label="Train",
color="#4C72B0")
        if val_col and val_col in res_df.columns:
            ax.plot(res_df["epoch"], res_df[val_col], label="Val /
mAP50-95",
                    color="#DD8452", linestyle="--")
        ax.set_title(f"RGB — {title}")
        ax.set_xlabel("Epoch")
        ax.grid(alpha=0.3)
        ax.legend(fontsize=8)

    plt.suptitle("RGB Model Training Curves", fontsize=14, y=1.01)
    plt.tight_layout()
    plt.savefig("rgb_training_curves.png", dpi=150,
bbox_inches="tight")
    plt.show()
else:
    print("results.csv not found — run training first.")
```



---

6 — Thermal Model Training

6.1 Thermal Imaging Differences

Thermal images encode temperature rather than reflected light. The key differences from RGB that affect training are:

Property	RGB	Thermal
Resolution	640 × 640	640 × 512
Channels	3 (R, G, B)	1 (brightness = temperature), false-coloured to 3ch
Texture detail	High	Low — edges are temperature gradients
Key feature	Surface appearance	Heat emission pattern
Hotspot detection	Poor	Excellent

Because the thermal images are stored as false-colour JPEGs (iron / rainbow palette), YOLOv8 can consume them directly as 3-channel inputs. The colour gradients carry temperature information through the palette mapping.

6.2 Training Configuration

The thermal model uses the same YOLOv8s backbone but with 8 output classes instead of 6. The input size is adjusted to 640 to match the YOLO stride requirements (even if source images are 640 × 512, they are padded during training).

```
# ── Write dataset YAML for Thermal ──────────────────────────────
thermal_yaml_content = {
    "path": str(THERMAL_DIR.resolve()),
    "train": "train/images",
    "val"  : "val/images",
    "test" : "test/images",
    "nc"   : 8,
    "names": THERMAL_CLASSES,
}

thermal_yaml_path = BASE_DIR / "thermal_dataset.yaml"
with open(thermal_yaml_path, "w") as f:
    yaml.dump(thermal_yaml_content, f, default_flow_style=False)

print(f"Thermal dataset YAML written to: {thermal_yaml_path}")
```

```
Thermal dataset YAML written to: thermal_dataset.yaml
```

```
# ── Train Thermal Model ─────────────────────────────────────────
thermal_model = YOLO("yolov8s.pt")

thermal_results = thermal_model.train(
    data    = str(thermal_yaml_path),
    epochs  = 100,
    imgsz   = 640,
    batch   = 16,
    device  = DEVICE,
    project = "runs/thermal",
    name    = "tower_thermal_v1",
    patience= 20,
    seed    = SEED,
    exist_ok= True,
    verbose = True,
)

THERMAL_BEST_WEIGHTS =
Path("runs/thermal/tower_thermal_v1/weights/best.pt")
print(f"\nBest thermal weights saved at: {THERMAL_BEST_WEIGHTS}")
```

```
Ultralytics 8.4.69 □ Python-3.12.3 torch-2.10.0+rocm7.2.4.git3d3aa833
CUDA:0 (AMD Instinct MI300X, 196592MiB)
engine/trainer: agnostic_nms=False, amp=True, angle=1.0,
augment=False, auto_augment=randaugment, batch=16, bgr=0.0, box=7.5,
cache=False, cfg=None, classes=None, close_mosaic=10, cls=0.5,
cls_pw=0.0, compile=False, conf=None, copy_paste=0.0,
copy_paste_mode=flip, cos_lr=False, cutmix=0.0,
data=thermal_dataset.yaml, degrees=0.0, deterministic=True, device=0,
dfl=1.5, dnn=False, dropout=0.0, dynamic=False, embed=None,
```

```
end2end=None, epochs=100, erasing=0.4, exist_ok=True, fliplr=0.5,
flipud=0.0, format=torchscript, fraction=1.0, freeze=None, half=False,
hsv_h=0.015, hsv_s=0.7, hsv_v=0.4, imgsz=640, int8=False, iou=0.7,
keras=False, kobj=1.0, line_width=None, lr0=0.01, lrf=0.01,
mask_ratio=4, max_det=300, mixup=0.0, mode=train, model=yolov8s.pt,
momentum=0.937, mosaic=1.0, multi_scale=0.0, name=tower_thermal_v1,
nbs=64, nms=False, opset=None, optimize=False, optimizer=auto,
overlap_mask=True, patience=20, perspective=0.0, plots=True,
pose=12.0, pretrained=True, profile=False, project=runs/thermal,
rect=False, resume=False, retina_masks=False, rle=1.0, save=True,
save_conf=False, save_crop=False,
save_dir=/workspace/shared/Test/Test2/runs/detect/runs/thermal/tower_t
hermal_v1, save_frames=False, save_json=False, save_period=-1,
save_txt=False, scale=0.5, seed=42, shear=0.0, show=False,
show_boxes=True, show_conf=True, show_labels=True, simplify=True,
single_cls=False, source=None, split=val, stream_buffer=False,
task=detect, time=None, tracker=botsort.yaml, translate=0.1, val=True,
verbose=True, vid_stride=1, visualize=False, warmup_bias_lr=0.1,
warmup_epochs=3.0, warmup_momentum=0.8, weight_decay=0.0005,
workers=8, workspace=None
Overriding model.yaml nc=80 with nc=8

                   from  n    params  module
arguments
  0                  -1  1       928  ultralytics.nn.modules.conv.Conv
[3, 32, 3, 2]
  1                  -1  1     18560  ultralytics.nn.modules.conv.Conv
[32, 64, 3, 2]
  2                  -1  1     29056  ultralytics.nn.modules.block.C2f
[64, 64, 1, True]
  3                  -1  1     73984  ultralytics.nn.modules.conv.Conv
[64, 128, 3, 2]
  4                  -1  2    197632  ultralytics.nn.modules.block.C2f
[128, 128, 2, True]
  5                  -1  1    295424  ultralytics.nn.modules.conv.Conv
[128, 256, 3, 2]
  6                  -1  2    788480  ultralytics.nn.modules.block.C2f
[256, 256, 2, True]
  7                  -1  1   1180672  ultralytics.nn.modules.conv.Conv
[256, 512, 3, 2]
  8                  -1  1   1838080  ultralytics.nn.modules.block.C2f
[512, 512, 1, True]
  9                  -1  1    656896
ultralytics.nn.modules.block.SPPF            [512, 512, 5]

 10                  -1  1         0
torch.nn.modules.upsampling.Upsample         [None, 2, 'nearest']

 11             [-1, 6]  1         0
ultralytics.nn.modules.conv.Concat           [1]
```

```
 12                  -1  1    591360  ultralytics.nn.modules.block.C2f
[768, 256, 1]
 13                  -1  1         0
torch.nn.modules.upsampling.Upsample         [None, 2, 'nearest']

 14             [-1, 4]  1         0
ultralytics.nn.modules.conv.Concat           [1]

 15                  -1  1    148224  ultralytics.nn.modules.block.C2f
[384, 128, 1]
 16                  -1  1    147712  ultralytics.nn.modules.conv.Conv
[128, 128, 3, 2]
 17            [-1, 12]  1         0
ultralytics.nn.modules.conv.Concat           [1]

 18                  -1  1    493056  ultralytics.nn.modules.block.C2f
[384, 256, 1]
 19                  -1  1    590336  ultralytics.nn.modules.conv.Conv
[256, 256, 3, 2]
 20             [-1, 9]  1         0
ultralytics.nn.modules.conv.Concat           [1]

 21                  -1  1   1969152  ultralytics.nn.modules.block.C2f
[768, 512, 1]
 22        [15, 18, 21]  1   2119144
ultralytics.nn.modules.head.Detect           [8, 16, None, [128, 256,
512]]
Model summary: 130 layers, 11,138,696 parameters, 11,138,680
gradients, 28.7 GFLOPs

Transferred 349/355 items from pretrained weights
Freezing layer 'model.22.dfl.conv.weight'
AMP: running Automatic Mixed Precision (AMP) checks...
AMP: checks passed 
train: Fast image access  (ping: 0.0±0.0 ms, read: 146.2±41.6 MB/s,
size: 74.0 KB)
train: Scanning
/workspace/shared/Test/Test2/cell_tower_thermal/train/labels.cache...
70 images, 0 backgrounds, 0 corrupt: 100% ━━━━━━━━━━━━ 70/70 24.5Mit/s
0.0s
val: Fast image access  (ping: 0.1±0.1 ms, read: 124.5±25.7 MB/s,
size: 55.2 KB)
val: Scanning
/workspace/shared/Test/Test2/cell_tower_thermal/val/labels.cache... 20
images, 0 backgrounds, 0 corrupt: 100% ━━━━━━━━━━━━ 20/20 1.1Mit/s
0.0s
optimizer: 'optimizer=auto' found, ignoring 'lr0=0.01' and
'momentum=0.937' and determining best 'optimizer', 'lr0' and
'momentum' automatically...
```

```
optimizer: AdamW(lr=0.000833, momentum=0.9) with parameter groups 57
weight(decay=0.0), 64 weight(decay=0.0005), 63 bias(decay=0.0)
Plotting labels to
/workspace/shared/Test/Test2/runs/detect/runs/thermal/tower_thermal_v1
/labels.jpg...
Image sizes 640 train, 640 val
Using 8 dataloader workers
Logging results to
/workspace/shared/Test/Test2/runs/detect/runs/thermal/tower_thermal_v1
Starting training for 100 epochs...

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 2.1s/it 2.1s
                   all         20        253      0.158     0.0656
0.0167    0.00382

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 10.4it/s 0.1s
                   all         20        253      0.641      0.135
0.108     0.0496

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 10.8it/s 0.1s
                   all         20        253      0.394      0.343
0.38       0.24

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 9.6it/s 0.1s
                   all         20        253      0.697       0.39
0.467      0.301

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 9.8it/s 0.1s
                   all         20        253      0.516      0.447
0.546      0.339

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 9.3it/s 0.1s
```

```
                   all         20        253      0.533      0.582
0.544      0.345

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 11.3it/s 0.1s
                   all         20        253      0.548      0.636
0.59      0.393

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 15.3it/s 0.1s
                   all         20        253      0.537       0.68
0.61      0.398

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 16.0it/s 0.1s
                   all         20        253      0.674      0.554
0.629      0.395

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 13.3it/s 0.1s
                   all         20        253      0.519       0.73
0.648       0.42

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 15.3it/s 0.1s
                   all         20        253      0.653      0.562
0.676      0.428

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 14.5it/s 0.1s
                   all         20        253      0.559      0.701
0.671      0.445

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 14.0it/s 0.1s
                   all         20        253      0.639      0.642
```

```
0.698      0.473

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 15.1it/s 0.1s
                   all         20        253      0.631      0.666
0.727      0.473

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 16.4it/s 0.1s
                   all         20        253      0.657      0.707
0.718      0.483

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 15.3it/s 0.1s
                   all         20        253      0.662      0.741
0.722      0.483

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 16.4it/s 0.1s
                   all         20        253      0.629       0.77
0.754      0.519

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 17.7it/s 0.1s
                   all         20        253      0.616       0.75
0.764      0.518

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 17.6it/s 0.1s
                   all         20        253      0.667      0.764
0.751      0.487

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 16.4it/s 0.1s
                   all         20        253       0.73      0.732
0.771      0.522
```

```
      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 16.7it/s 0.1s
                   all         20        253       0.66      0.753
0.756      0.538

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 15.1it/s 0.1s
                   all         20        253      0.743      0.726
0.753      0.534

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 17.2it/s 0.1s
                   all         20        253      0.755      0.766
0.782      0.543

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 17.6it/s 0.1s
                   all         20        253      0.749      0.831
0.817      0.573

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 17.4it/s 0.1s
                   all         20        253       0.82      0.792
0.819      0.557

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 15.4it/s 0.1s
                   all         20        253       0.83       0.79
0.845      0.575

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 17.4it/s 0.1s
                   all         20        253      0.775      0.822
0.822      0.576
```

```
      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 17.2it/s 0.1s
                   all         20        253      0.811      0.808
0.837      0.559

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 16.5it/s 0.1s
                   all         20        253      0.801      0.769
0.818      0.577

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 17.8it/s 0.1s
                   all         20        253      0.822      0.753
0.832      0.608

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 18.1it/s 0.1s
                   all         20        253      0.826      0.789
0.849      0.606

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 18.2it/s 0.1s
                   all         20        253      0.833      0.814
0.84      0.613

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 18.0it/s 0.1s
                   all         20        253      0.812       0.79
0.821      0.615

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 17.5it/s 0.1s
                   all         20        253       0.81      0.788
0.817      0.617

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
```

```
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 18.7it/s 0.1s
                   all         20        253      0.833        0.8
0.818      0.618

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 17.5it/s 0.1s
                   all         20        253      0.824      0.833
0.836      0.629

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 18.9it/s 0.1s
                   all         20        253      0.849      0.836
0.856      0.643

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 18.1it/s 0.1s
                   all         20        253       0.85      0.822
0.85      0.645

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 19.1it/s 0.1s
                   all         20        253      0.828       0.81
0.85      0.624

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 16.3it/s 0.1s
                   all         20        253      0.839      0.827
0.852      0.644

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 18.3it/s 0.1s
                   all         20        253      0.879      0.805
0.853      0.653

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
```

```
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 18.5it/s 0.1s
                   all         20        253      0.854      0.787
0.84      0.636

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 18.1it/s 0.1s
                   all         20        253      0.871      0.779
0.845      0.636

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 19.3it/s 0.1s
                   all         20        253      0.867      0.819
0.864       0.66

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 15.1it/s 0.1s
                   all         20        253      0.872      0.815
0.863      0.661

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 17.1it/s 0.1s
                   all         20        253      0.843      0.835
0.856      0.652

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 18.9it/s 0.1s
                   all         20        253      0.894      0.802
0.864      0.658

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 16.4it/s 0.1s
                   all         20        253      0.898      0.806
0.85       0.65

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
```

```
━━━━━━━━━━━━ 1/1 20.4it/s 0.0s
                   all         20        253      0.869      0.828
0.857      0.656

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 17.6it/s 0.1s
                   all         20        253      0.875      0.844
0.868      0.664

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 18.9it/s 0.1s
                   all         20        253      0.875      0.848
0.874      0.669

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 20.4it/s 0.0s
                   all         20        253      0.917      0.815
0.865      0.664

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 18.9it/s 0.1s
                   all         20        253      0.893      0.833
0.868      0.668

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 20.6it/s 0.0s
                   all         20        253      0.891      0.828
0.866      0.665

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 18.2it/s 0.1s
                   all         20        253      0.878      0.816
0.853      0.654

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 18.9it/s 0.1s
```

```
                   all         20        253      0.912      0.822
0.859      0.666

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 20.5it/s 0.0s
                   all         20        253      0.907      0.823
0.861      0.672

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 16.8it/s 0.1s
                   all         20        253      0.917      0.822
0.858      0.678

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 16.9it/s 0.1s
                   all         20        253      0.903      0.824
0.858      0.677

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 19.2it/s 0.1s
                   all         20        253      0.899      0.831
0.865      0.679

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 18.8it/s 0.1s
                   all         20        253      0.895      0.828
0.875      0.687

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 18.8it/s 0.1s
                   all         20        253      0.871      0.836
0.875      0.681

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 18.8it/s 0.1s
                   all         20        253      0.866      0.841
```

```
0.875      0.678

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 18.6it/s 0.1s
                   all         20        253      0.907      0.825
0.874      0.677

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 20.6it/s 0.0s
                   all         20        253      0.905      0.813
0.869      0.662

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 18.6it/s 0.1s
                   all         20        253      0.894      0.826
0.872      0.653

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 18.8it/s 0.1s
                   all         20        253      0.901      0.831
0.874      0.644

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 18.6it/s 0.1s
                   all         20        253      0.907      0.831
0.876      0.663

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 18.0it/s 0.1s
                   all         20        253      0.878      0.845
0.878       0.68

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 15.9it/s 0.1s
                   all         20        253       0.89      0.823
0.88      0.666
```

```
      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 19.2it/s 0.1s
                   all         20        253       0.93      0.806
0.874      0.663

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 18.8it/s 0.1s
                   all         20        253      0.863      0.848
0.874      0.682

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 19.7it/s 0.1s
                   all         20        253        0.9      0.828
0.879      0.694

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 19.1it/s 0.1s
                   all         20        253      0.851      0.865
0.88      0.694

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 16.7it/s 0.1s
                   all         20        253      0.877      0.849
0.886      0.692

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 19.9it/s 0.1s
                   all         20        253        0.9      0.828
0.88      0.694

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 19.5it/s 0.1s
                   all         20        253       0.92      0.837
0.878      0.708
```

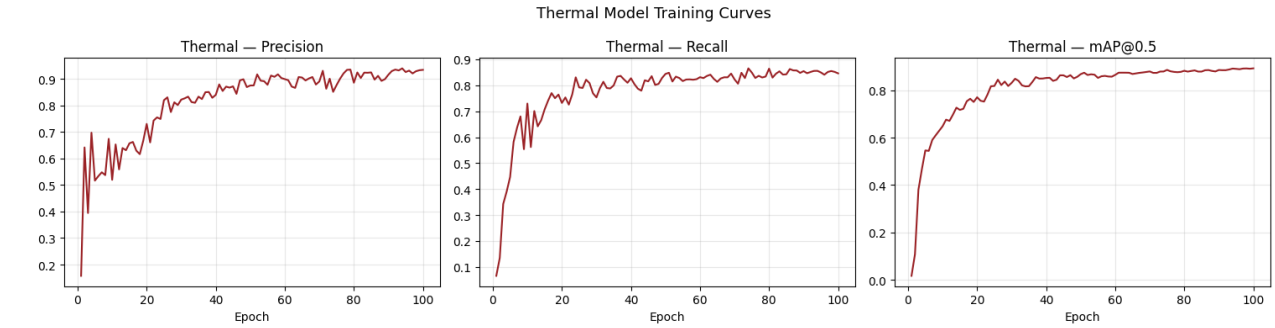
```
      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 17.5it/s 0.1s
                   all         20        253      0.934       0.83
0.877      0.705

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 17.7it/s 0.1s
                   all         20        253      0.935      0.834
0.878       0.71

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 17.9it/s 0.1s
                   all         20        253      0.885      0.864
0.883      0.711

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 19.2it/s 0.1s
                   all         20        253      0.924       0.83
0.879      0.711

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 17.9it/s 0.1s
                   all         20        253      0.903      0.845
0.882      0.711

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 19.1it/s 0.1s
                   all         20        253      0.923      0.854
0.884      0.709

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 19.9it/s 0.1s
                   all         20        253      0.923      0.841
0.879      0.701

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
```

```
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 17.9it/s 0.1s
                   all         20        253      0.924      0.842
0.879      0.697

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 16.8it/s 0.1s
                   all         20        253      0.897      0.863
0.885      0.698

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 20.3it/s 0.0s
                   all         20        253      0.911      0.858
0.886      0.704

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 16.2it/s 0.1s
                   all         20        253      0.892      0.857
0.882      0.702

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 19.7it/s 0.1s
                   all         20        253      0.899      0.848
0.88        0.7

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 18.2it/s 0.1s
                   all         20        253      0.915      0.855
0.886      0.703
Closing dataloader mosaic

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 12.8it/s 0.1s
                   all         20        253      0.929      0.847
0.885        0.7

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
```

```
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 17.3it/s 0.1s
                   all         20        253      0.935      0.851
0.885      0.704

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 19.3it/s 0.1s
                   all         20        253      0.932      0.856
0.888      0.702

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 17.6it/s 0.1s
                   all         20        253      0.939      0.856
0.892      0.708

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 17.3it/s 0.1s
                   all         20        253      0.925       0.85
0.891      0.706

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 16.3it/s 0.1s
                   all         20        253      0.931      0.841
0.889      0.707

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 17.1it/s 0.1s
                   all         20        253       0.92      0.851
0.892      0.714

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 18.1it/s 0.1s
                   all         20        253      0.929      0.855
0.893      0.713

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
```

```
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 16.1it/s 0.1s
                   all         20        253      0.932      0.852
0.891      0.715

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances
Size
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 19.4it/s 0.1s
                   all         20        253      0.934      0.846
0.893       0.72

100 epochs completed in 0.035 hours.
Optimizer stripped from
/workspace/shared/Test/Test2/runs/detect/runs/thermal/tower_thermal_v1
/weights/last.pt, 22.5MB
Optimizer stripped from
/workspace/shared/Test/Test2/runs/detect/runs/thermal/tower_thermal_v1
/weights/best.pt, 22.5MB

Validating
/workspace/shared/Test/Test2/runs/detect/runs/thermal/tower_thermal_v1
/weights/best.pt...
Ultralytics 8.4.69 □ Python-3.12.3 torch-2.10.0+rocm7.2.4.git3d3aa833
CUDA:0 (AMD Instinct MI300X, 196592MiB)
Model summary (fused): 73 layers, 11,128,680 parameters, 0 gradients,
28.5 GFLOPs
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 17.6it/s 0.1s
                   all         20        253      0.938      0.843
0.891      0.719
       tower_structure         16         16       0.99          1
0.995      0.872
         antenna_panel         19         72      0.998      0.944
0.968       0.86
         cable_harness         20         44      0.837      0.585
0.667      0.406
       hotspot_critical         14         18      0.918          1
0.989      0.867
       hotspot_moderate         14         17      0.933      0.706
0.876      0.675
         hotspot_minor         11         14      0.887      0.786
0.869      0.567
        connector_joint         20         57      0.976      0.722
0.771       0.53
      equipment_cabinet         11         15      0.968          1
0.995      0.974
Speed: 0.0ms preprocess, 0.4ms inference, 0.0ms loss, 0.5ms
postprocess per image
Results saved to
```

```
/workspace/shared/Test/Test2/runs/detect/runs/thermal/tower_thermal_v1

Best thermal weights saved at:
runs/thermal/tower_thermal_v1/weights/best.pt

# ── Thermal training curves ──────────────────────────────────────
th_results_csv =
Path("runs/detect/runs/thermal/tower_thermal_v1/results.csv")

if th_results_csv.exists():
    th_res_df = pd.read_csv(th_results_csv)
    th_res_df.columns = th_res_df.columns.str.strip()

    fig, axes = plt.subplots(1, 3, figsize=(15, 4))
    for (col, label), ax in zip(
        [("metrics/precision(B)", "Precision"),
         ("metrics/recall(B)",    "Recall"),
         ("metrics/mAP50(B)",     "mAP@0.5")],
        axes
    ):
        if col in th_res_df.columns:
            ax.plot(th_res_df["epoch"], th_res_df[col],
color="#9b2226")
        ax.set_title(f"Thermal — {label}")
        ax.set_xlabel("Epoch")
        ax.grid(alpha=0.3)

    plt.suptitle("Thermal Model Training Curves", fontsize=13)
    plt.tight_layout()
    plt.savefig("thermal_training_curves.png", dpi=150,
bbox_inches="tight")
    plt.show()
else:
    print("Thermal results.csv not found — run training first.")
```



7 — Fusion Layer — Theory & Implementation

7.1 The Fusion Problem

Three independent data streams arrive at the same time for every drone frame:

- **RGB detection** — a list of bounding boxes with class labels and confidence scores
- **Thermal detection** — a separate list of bounding boxes from the thermal model
- **Telemetry** — a single row of 63 numerical fields describing drone state and environment

The goal of the fusion layer is to combine these into a single ranked list of anomaly events, each with a unified confidence score and severity label.

7.2 Fusion Strategy: Weighted Late Fusion

Late fusion combines the *outputs* of independently trained models (as opposed to early fusion, which concatenates raw inputs, or feature-level fusion, which merges internal representations). Late fusion is chosen here because:

1. The RGB and Thermal models have different input formats and class sets.
2. Telemetry has no spatial structure (it is a time-series scalar), making feature-level fusion impractical.
3. Late fusion allows each modality to be trained, updated, and replaced independently.

7.3 Fusion Formula

For each frame t, the fused confidence score S_{fused} for a detected anomaly is:

$$S_{fused} = \alpha \cdot c_{rgb} \cdot w_{rgb} + \beta \cdot c_{th} \cdot w_{th} + \gamma \cdot c_{telem} \cdot w_{telem}$$

subject to $\alpha + \beta + \gamma = 1$.

Symbol	Meaning
c_{rgb}	Detection confidence from the RGB model (0–1)
c_{th}	Detection confidence from the Thermal model (0–1)
c_{telem}	Anomaly confidence from the Telemetry field (0–1)
w_{rgb}	Severity weight of the RGB class (see table in §2)
w_{th}	Severity weight of the Thermal class
w_{telem}	Severity weight of the Telemetry class
$\alpha = 0.45$	Modality weight for RGB
$\beta = 0.40$	Modality weight for Thermal
$\gamma = 0.15$	Modality weight for Telemetry

Why these weights? RGB detections are the primary source of structural defects (cracks, rust, bolts). Thermal detections are highly reliable for hotspot events. Telemetry serves as a contextual signal — vibration, wind, and altitude affect inspection quality, so it carries less direct anomaly weight but boosts overall confidence when the telemetry anomaly class agrees.

7.4 Cross-Modal Semantic Mapping

Classes across modalities overlap semantically but not exactly. The table below defines the mapping used for agreement boosting:

RGB Class	Thermal Class	Telemetry Class	Semantic Group
corrosion	—	corrosion	Corrosion
crack	—	structural_crack	Structural damage
structural_bend	—	structural_crack	Structural damage
—	hotspot_critical	hotspot_critical, cable_fault, connector_failure	Electrical/thermal fault
—	hotspot_moderate	hotspot_moderate	Thermal anomaly
missing_bolt	—	—	Fastener issue

When two or more modalities agree on the same semantic group for a given frame, a **bonus multiplier** of 1.15 is applied to S_{fused}.

7.5 Telemetry Context Extraction

Not all 63 telemetry fields are equally relevant. The following fields are extracted as a context vector z:

$$z = [\text{altitude\_m}, \text{wind\_speed}, \text{vibration\_level}, \text{hdop}, \text{battery\_pct}, \text{ir\_temp\_delta}]$$

A **reliability factor** r is computed from the GPS horizontal dilution of precision (HDOP):

$$r = max\left(0, 1 - \frac{\text{hdop} - 1}{5}\right)$$

When HDOP is 1.0 (ideal GPS), $r = 1.0$. When HDOP reaches 6.0 (degraded), $r = 0.0$. The telemetry anomaly confidence is then:

$$c_{telem} = r \cdot c_{telem,raw}$$

This ensures that detections in poor GPS conditions are downweighted automatically.

```
# ── Fusion Layer Implementation ──────────────────────────────

# Modality weights
ALPHA = 0.45   # RGB
BETA  = 0.40   # Thermal
GAMMA = 0.15   # Telemetry
```

```
# Semantic group mappings
RGB_TO_GROUP = {
    "corrosion"       : "corrosion",
    "crack"           : "structural",
    "structural_bend" : "structural",
    "missing_bolt"    : "fastener",
    "rust"            : "surface",
    "paint_damage"    : "surface",
}
THERMAL_TO_GROUP = {
    "hotspot_critical" : "thermal_fault",
    "hotspot_moderate" : "thermal_anomaly",
    "hotspot_minor"    : "thermal_minor",
    "cable_harness"    : "cable",
    "connector_joint"  : "connector",
}
TELEM_TO_GROUP = {
    "hotspot_critical"   : "thermal_fault",
    "cable_fault"        : "thermal_fault",
    "connector_failure"  : "thermal_fault",
    "hotspot_moderate"   : "thermal_anomaly",
    "corrosion"          : "corrosion",
    "structural_crack"   : "structural",
    "antenna_misalignment": "structural",
}

AGREEMENT_BONUS = 1.15


def compute_reliability(hdop: float) -> float:
    """GPS reliability factor based on HDOP.

    r = max(0, 1 - (hdop - 1) / 5)
    Returns 1.0 for hdop=1 (ideal), 0.0 for hdop>=6.
    """
    return max(0.0, 1.0 - (hdop - 1.0) / 5.0)


def fuse_frame(
    rgb_detections  : list,   # [{class, confidence, bbox}]
    thermal_detections: list, # [{class, confidence, bbox}]
    telem_row       : dict,   # single CSV row as dict
) -> list:
    """
    Merge RGB + Thermal + Telemetry signals for one frame.
    Returns a list of fused anomaly records sorted by fused_score
descending.

    Each record:
```

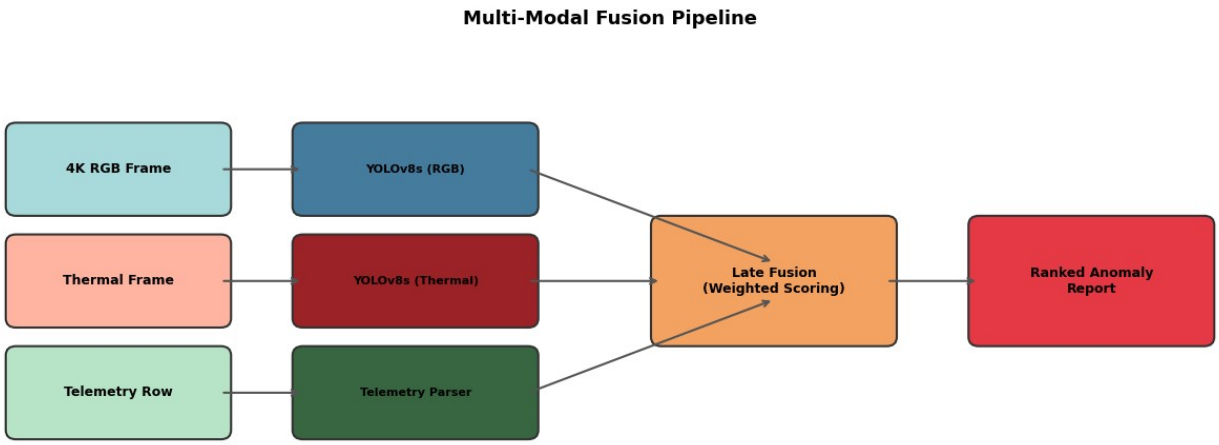
```
        {source, class, confidence, fused_score, severity_weight,
group, bbox}
    """
    # ── Telemetry confidence ──────────────────────────────────────
    hdop           = float(telem_row.get("hdop", 1.0))
    reliability    = compute_reliability(hdop)
    telem_class    = str(telem_row.get("class", "no_anomaly"))
    telem_conf_raw = float(telem_row.get("confidence", 0.0))
    c_telem        = reliability * telem_conf_raw
    w_telem        = SEVERITY_WEIGHT.get(telem_class, 0.0)
    telem_group    = TELEM_TO_GROUP.get(telem_class, "none")

    fused_records = []

    # ── RGB detections ────────────────────────────────────────────
    for det in rgb_detections:
        cls      = det["class"]
        c_rgb    = det["confidence"]
        w_rgb    = SEVERITY_WEIGHT.get(cls, 0.3)
        rgb_grp  = RGB_TO_GROUP.get(cls, "unknown")

        # Agreement check
        bonus = AGREEMENT_BONUS if (rgb_grp == telem_group) else 1.0

        score = (ALPHA * c_rgb * w_rgb
                 + GAMMA * c_telem * w_telem) * bonus

        fused_records.append({
            "source"         : "rgb",
            "class"          : cls,
            "confidence"     : round(c_rgb, 4),
            "fused_score"    : round(min(score, 1.0), 4),
            "severity_weight": w_rgb,
            "group"          : rgb_grp,
            "bbox"           : det.get("bbox", []),
        })

    # ── Thermal detections ────────────────────────────────────────
    for det in thermal_detections:
        cls      = det["class"]
        c_th     = det["confidence"]
        w_th     = SEVERITY_WEIGHT.get(cls, 0.3)
        th_grp   = THERMAL_TO_GROUP.get(cls, "unknown")

        bonus = AGREEMENT_BONUS if (th_grp == telem_group) else 1.0

        score = (BETA * c_th * w_th
```

```
                + GAMMA * c_telem * w_telem) * bonus

        fused_records.append({
            "source"         : "thermal",
            "class"          : cls,
            "confidence"     : round(c_th, 4),
            "fused_score"    : round(min(score, 1.0), 4),
            "severity_weight": w_th,
            "group"          : th_grp,
            "bbox"           : det.get("bbox", []),
        })

    # Sort by fused_score descending
    fused_records.sort(key=lambda x: x["fused_score"], reverse=True)
    return fused_records


# ── Quick smoke test ──────────────────────────────────────────────────
test_rgb  = [{"class": "crack",          "confidence": 0.82, "bbox":
[0.4, 0.5, 0.1, 0.2]}]
test_th   = [{"class": "hotspot_critical","confidence": 0.91, "bbox":
[0.4, 0.5, 0.15, 0.2]}]
test_telem = {"hdop": 1.2, "class": "structural_crack", "confidence":
0.78}

result = fuse_frame(test_rgb, test_th, test_telem)
print("Fusion smoke test — top results:")
for r in result:
    print(f"  [{r['source']:8s}] {r['class']:20s}
conf={r['confidence']:.2f}  "
          f"fused={r['fused_score']:.4f}  group={r['group']}")
```

```
Fusion smoke test — top results:
  [rgb     ] crack                conf=0.82  fused=0.4834
group=structural
  [thermal ] hotspot_critical     conf=0.91  fused=0.4525
group=thermal_fault
```

```
# ── Visualise the fusion pipeline as a block diagram ──────────────────
fig, ax = plt.subplots(figsize=(12, 5))
ax.set_xlim(0, 12)
ax.set_ylim(0, 5)
ax.axis("off")

def draw_box(ax, x, y, w, h, label, color, fontsize=9):
    box = mpatches.FancyBboxPatch((x, y), w, h,
                                  boxstyle="round,pad=0.1",
                                  linewidth=1.5, edgecolor="#333",
```

```
                                    facecolor=color, zorder=2)
    ax.add_patch(box)
    ax.text(x + w/2, y + h/2, label, ha="center", va="center",
            fontsize=fontsize, fontweight="bold", wrap=True)

def arrow(ax, x1, y1, x2, y2):
    ax.annotate("", xy=(x2, y2), xytext=(x1, y1),
                arrowprops=dict(arrowstyle="->", color="#555",
lw=1.5))

# Input blocks
draw_box(ax, 0.2, 3.2, 2.0, 0.8, "4K RGB Frame",     "#a8dadc")
draw_box(ax, 0.2, 2.0, 2.0, 0.8, "Thermal Frame",     "#ffb4a2")
draw_box(ax, 0.2, 0.8, 2.0, 0.8, "Telemetry Row",     "#b7e4c7")

# Model blocks
draw_box(ax, 3.0, 3.2, 2.2, 0.8, "YOLOv8s (RGB)",     "#457b9d",
fontsize=8)
draw_box(ax, 3.0, 2.0, 2.2, 0.8, "YOLOv8s (Thermal)", "#9b2226",
fontsize=8)
draw_box(ax, 3.0, 0.8, 2.2, 0.8, "Telemetry Parser",   "#386641",
fontsize=8)

# Fusion block
draw_box(ax, 6.5, 1.8, 2.2, 1.2, "Late Fusion\n(Weighted Scoring)",
"#f4a261", fontsize=9)

# Output block
draw_box(ax, 9.6, 1.8, 2.2, 1.2, "Ranked Anomaly\nReport", "#e63946",
fontsize=9)

# Arrows — inputs to models
arrow(ax, 2.2, 3.6, 3.0, 3.6)
arrow(ax, 2.2, 2.4, 3.0, 2.4)
arrow(ax, 2.2, 1.2, 3.0, 1.2)

# Arrows — models to fusion
arrow(ax, 5.2, 3.6, 7.6, 2.6)
arrow(ax, 5.2, 2.4, 6.5, 2.4)
arrow(ax, 5.2, 1.2, 7.6, 2.2)

# Fusion to output
arrow(ax, 8.7, 2.4, 9.6, 2.4)

ax.set_title("Multi-Modal Fusion Pipeline", fontsize=13,
fontweight="bold", pad=10)
plt.tight_layout()
plt.savefig("fusion_pipeline_diagram.png", dpi=150,
bbox_inches="tight")
plt.show()
```



---

8 — Evaluation & Metrics

8.1 Detection Metrics Defined

Intersection over Union (IoU)

$$\text{IoU} = \frac{\left|\hat{B} \cap B_{gt}\right)}{\left|\hat{B} \cup B_{gt}\right)}$$

A predicted box $\hat{B}$ is a true positive (TP) when $\text{IoU} \geq \theta$ (threshold, typically 0.5).

Precision and Recall

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

Average Precision (AP) and mAP

AP for one class is the area under the Precision–Recall curve:

$$AP = \int_0^1 P(R)\, dR \approx \sum_{k=1}^{N} P(k) \cdot \Delta R(k)$$

mAP@0.5 averages AP over all classes at IoU = 0.5. mAP@0.5:0.95 further averages over IoU thresholds from 0.5 to 0.95 in steps of 0.05, giving a stricter measure of localisation quality.

$$\text{mAP@0.5:0.95} = \frac{1}{10} \sum_{\theta=0.50}^{0.95} \text{mAP}(\theta)$$

F1 Score

$$F_1 = 2 \cdot \frac{P \cdot R}{P + R}$$

8.2 Running Validation

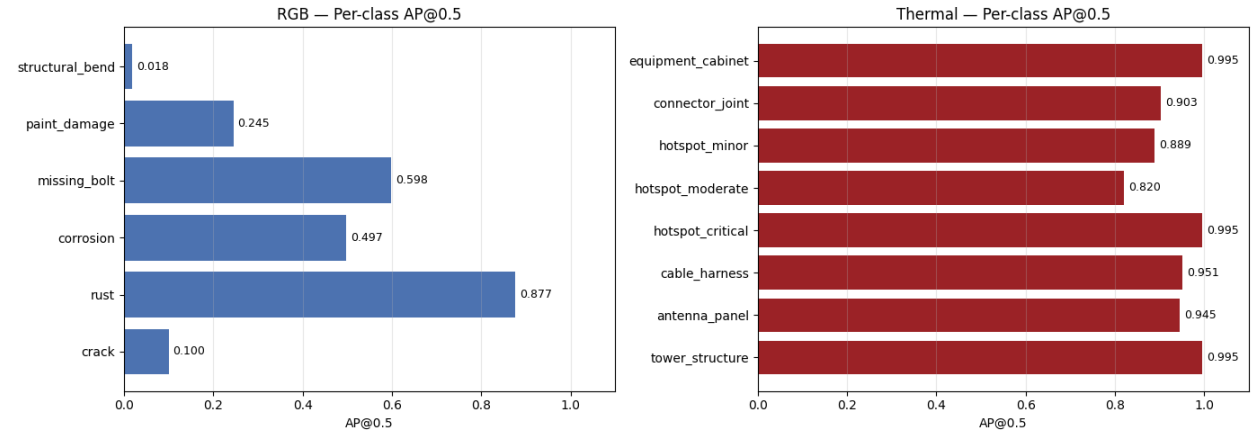
```
from pathlib import Path
from ultralytics import YOLO

# Set correct path
RGB_BEST_WEIGHTS =
Path("runs/detect/runs/rgb/tower_rgb_v1/weights/best.pt")

if RGB_BEST_WEIGHTS.exists():
    rgb_eval_model = YOLO(str(RGB_BEST_WEIGHTS))

    rgb_metrics = rgb_eval_model.val(
        data   = str(rgb_yaml_path),
        split  = "test",
        imgsz  = 640,
        device = DEVICE,
        verbose= True,
    )

    print("\n=== RGB Model — Test Metrics ===")
    print(f"  mAP@0.5      : {rgb_metrics.box.map50:.4f}")
    print(f"  mAP@0.5:0.95 : {rgb_metrics.box.map:.4f}")
    print(f"  Precision    : {rgb_metrics.box.mp:.4f}")
    print(f"  Recall       : {rgb_metrics.box.mr:.4f}")
else:
    print(f"RGB weights not found at: {RGB_BEST_WEIGHTS}")
```

```
Ultralytics 8.4.69 🚀 Python-3.12.3 torch-2.10.0+rocm7.2.4.git3d3aa833
CUDA:0 (AMD Instinct MI300X, 196592MiB)
Model summary (fused): 73 layers, 11,127,906 parameters, 0 gradients,
28.4 GFLOPs
val: Fast image access 🚀 (ping: 0.2±0.1 ms, read: 1630.9±324.1 MB/s,
size: 807.1 KB)
val: Scanning
/workspace/shared/Test/Test2/tower_inspection_dataset/test/labels.cach
e... 10 images, 0 backgrounds, 0 corrupt: 100% ━━━━━━━━━━━━ 10/10
4.7Mit/s 0.0s
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 1.9s/it 1.9s
                   all         10         36      0.651       0.45
0.389      0.318
                 crack          8         10      0.489        0.1
0.1     0.0765
                  rust          6          9      0.661          1
0.877      0.731
```

```
             corrosion          4          4      0.467       0.75
0.497      0.398
          missing_bolt          5          5      0.805        0.6
0.598      0.496
          paint_damage          4          4      0.486       0.25
0.245      0.196
       structural_bend          4          4          1          0
0.0185    0.00903
Speed: 0.3ms preprocess, 186.1ms inference, 0.0ms loss, 0.8ms
postprocess per image
Results saved to /workspace/shared/Test/Test2/runs/detect/val-5

=== RGB Model — Test Metrics ===
  mAP@0.5      : 0.3894
  mAP@0.5:0.95 : 0.3177
  Precision    : 0.6513
  Recall       : 0.4500

# ── Evaluate Thermal model on test split
─────────────────────────────────────────
from pathlib import Path
from ultralytics import YOLO

# Set correct path
THERMAL_BEST_WEIGHTS =
Path("runs/detect/runs/thermal/tower_thermal_v1/weights/best.pt")

if THERMAL_BEST_WEIGHTS.exists():
    th_eval_model = YOLO(str(THERMAL_BEST_WEIGHTS))

    th_metrics = th_eval_model.val(
        data   = str(thermal_yaml_path),
        split  = "test",
        imgsz  = 640,
        device = DEVICE,
        verbose= True,
    )

    print("\n=== Thermal Model — Test Metrics ===")
    print(f"  mAP@0.5      : {th_metrics.box.map50:.4f}")
    print(f"  mAP@0.5:0.95 : {th_metrics.box.map:.4f}")
    print(f"  Precision    : {th_metrics.box.mp:.4f}")
    print(f"  Recall       : {th_metrics.box.mr:.4f}")
else:
    print(f"Thermal weights not found at: {THERMAL_BEST_WEIGHTS}")

Ultralytics 8.4.69 □ Python-3.12.3 torch-2.10.0+rocm7.2.4.git3d3aa833
CUDA:0 (AMD Instinct MI300X, 196592MiB)
Model summary (fused): 73 layers, 11,128,680 parameters, 0 gradients,
28.5 GFLOPs
```

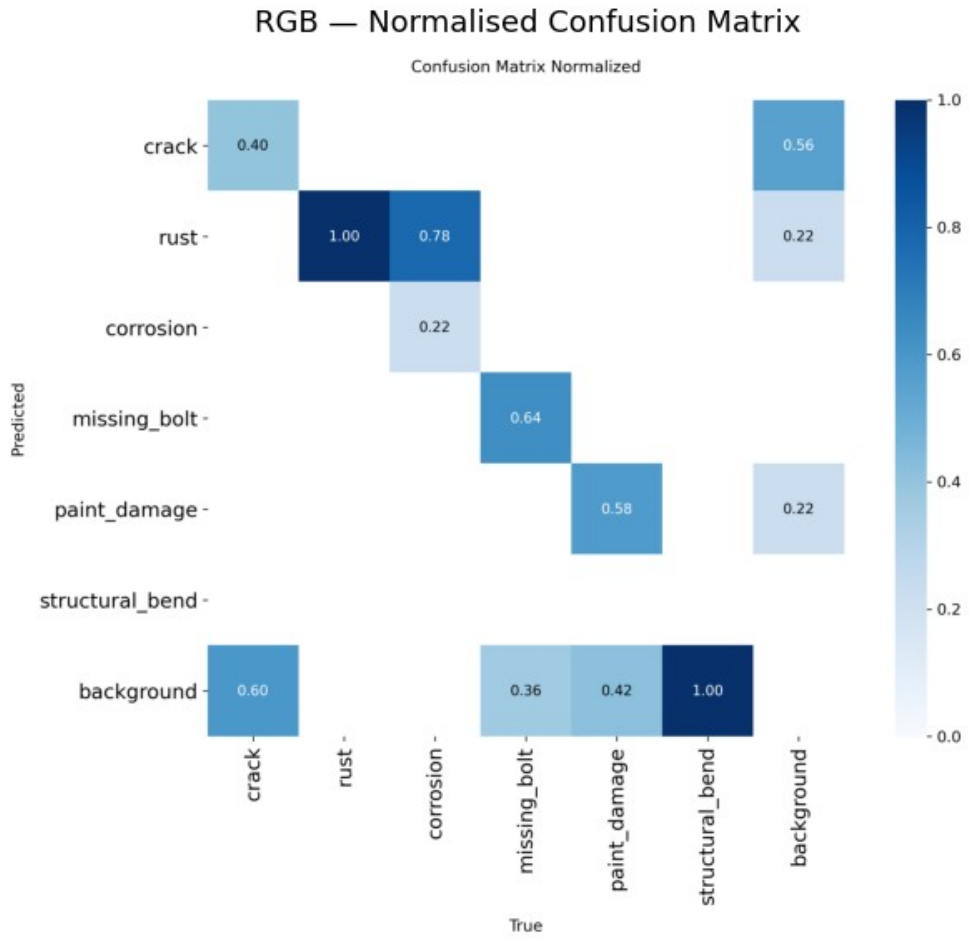
```
val: Fast image access  (ping: 0.1±0.2 ms, read: 257.8±107.8 MB/s,
size: 83.7 KB)
val: Scanning
/workspace/shared/Test/Test2/cell_tower_thermal/test/labels.cache...
10 images, 0 backgrounds, 0 corrupt: 100% ━━━━━━━━━━━━ 10/10 4.2Mit/s
0.0s
ages  Instances      Box(P          R      mAP50  mAP50-95): 100%
━━━━━━━━━━━━ 1/1 1.7s/it 1.7s
                   all         10        103      0.941      0.898
0.937      0.798
       tower_structure          4          4      0.944          1
0.995       0.92
         antenna_panel          6         20      0.986       0.95
0.945      0.928
         cable_harness         10         18      0.933      0.777
0.951      0.716
      hotspot_critical          9         10          1      0.927
0.995      0.888
      hotspot_moderate          5          8      0.829       0.75
0.82      0.568
        hotspot_minor          6          9      0.874      0.889
0.889      0.676
       connector_joint         10         18      0.984      0.889
0.903       0.69
     equipment_cabinet          8         16      0.976          1
0.995      0.995
Speed: 0.2ms preprocess, 159.9ms inference, 0.0ms loss, 0.4ms
postprocess per image
Results saved to /workspace/shared/Test/Test2/runs/detect/val-6

=== Thermal Model — Test Metrics ===
  mAP@0.5      : 0.9367
  mAP@0.5:0.95 : 0.7977
  Precision    : 0.9408
  Recall       : 0.8977

# ── Per-class AP comparison table
──────────────────────────────────────────────────
# This cell visualises per-class AP50 if metrics are available.

def plot_per_class_ap(metrics_obj, class_names, title, color, ax):
    """Plot per-class AP50 as a horizontal bar chart."""
    try:
        ap50_per_class = metrics_obj.box.ap50
        ax.barh(class_names, ap50_per_class, color=color)
        for i, v in enumerate(ap50_per_class):
            ax.text(v + 0.01, i, f"{v:.3f}", va="center", fontsize=9)
        ax.set_xlim(0, 1.1)
        ax.set_title(title)
        ax.set_xlabel("AP@0.5")
```

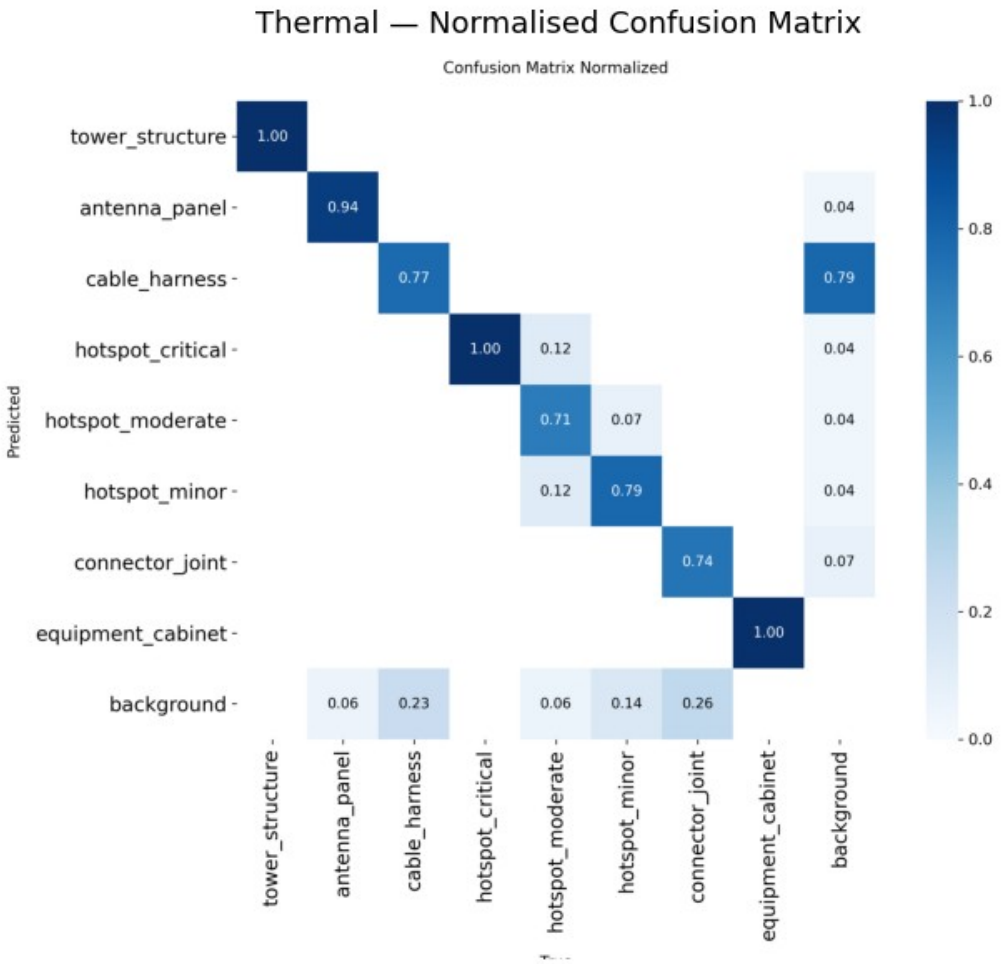
```
        ax.grid(axis="x", alpha=0.3)
    except Exception:
        ax.text(0.5, 0.5, "Run evaluation first",
                ha="center", va="center", transform=ax.transAxes)

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

try:
    plot_per_class_ap(rgb_metrics, RGB_CLASSES,     "RGB — Per-class
AP@0.5",     "#4C72B0", ax1)
    plot_per_class_ap(th_metrics,  THERMAL_CLASSES, "Thermal — Per-
class AP@0.5", "#9b2226", ax2)
except NameError:
    for ax in (ax1, ax2):
        ax.text(0.5, 0.5, "Metrics not yet computed\n(train first)",
                ha="center", va="center", transform=ax.transAxes,
fontsize=12)

plt.tight_layout()
plt.savefig("per_class_ap.png", dpi=150, bbox_inches="tight")
plt.show()
```




```
# ── Confusion matrices are saved by YOLO automatically
# ──────────────────────────────────────────────────

# Load and display from the training output directory

for label, cm_path in [
    ("RGB",
Path("runs/detect/runs/rgb/tower_rgb_v1/confusion_matrix_normalized.pn
g")),
    ("Thermal",
Path("runs/detect/runs/thermal/tower_thermal_v1/confusion_matrix_norma
lized.png")),
]:
    if cm_path.exists():
```

```
        img = plt.imread(str(cm_path))
        plt.figure(figsize=(8, 6))
        plt.imshow(img)
        plt.axis("off")
        plt.title(f"{label} — Normalised Confusion Matrix",
fontsize=13)
        plt.tight_layout()
        plt.show()
    else:
        print(f"Confusion matrix not found for {label} — will appear
after training.")
```





---

9 — Inference Replay on Video or Image

9.1 How This Works

Since in this hackathon we are not flying a real drone, the replay mode simulates what would happen on an actual mission:

1. The RGB model runs on each frame of the input video (or on the single image).
2. The Thermal model runs on the same frame — in a real mission this would be a synchronised thermal camera; during replay, we optionally accept a paired thermal video/image. If none is provided, the thermal stream is disabled and β redistributed to α.
3. A telemetry row is looked up (or synthesised) per frame using the `frame_id` key.
4. The fusion layer produces a ranked anomaly list for every frame.
5. Bounding boxes and fused scores are drawn on the output frame.
6. All anomaly records are exported to a CSV report.

9.2 Input Configuration

```
# ── CONFIGURE THESE PATHS ──────────────────────────────────────

INPUT_SOURCE    = "tower_0012.png"  # ← replace with your file path
THERMAL_SOURCE  = None                          # ← path to paired
thermal video/image,
                                                #   or None to skip
thermal stream
TELEMETRY_CSV   = None                          # ← mission CSV path, or
None
                                                #   (synthetic telemetry
used if None)
OUTPUT_DIR      = Path("replay_output")
OUTPUT_DIR.mkdir(exist_ok=True)

CONF_THRESHOLD  = 0.30   # detection confidence threshold
IOU_THRESHOLD   = 0.45   # NMS IoU threshold

# Default telemetry row used when no CSV is available
DEFAULT_TELEM_ROW = {
    "hdop"      : 1.5,
    "class"     : "no_anomaly",
    "confidence": 0.0,
    "altitude_m": 30.0,
    "wind_speed": 3.0,
}

print(f"Input source  : {INPUT_SOURCE}")
print(f"Thermal source: {THERMAL_SOURCE}")
print(f"Telemetry CSV : {TELEMETRY_CSV}")
print(f"Output dir    : {OUTPUT_DIR}")
```

```
Input source  : tower_0012.png
Thermal source: None
Telemetry CSV : None
Output dir    : replay_output
```

```
# ── Helper: load telemetry into a frame-indexed dict ──────────
def load_telemetry_lookup(csv_path):
    if csv_path is None or not Path(csv_path).exists():
        return None
    df = pd.read_csv(csv_path)
    if "frame_id" in df.columns:
        return df.set_index("frame_id").to_dict(orient="index")
    return None


# ── Helper: parse YOLO results into our dict format
```

```
# ──────────────────────────────────
def parse_yolo_results(result, class_names):
    detections = []
    if result.boxes is None:
        return detections
    for box in result.boxes:
        cls_id  = int(box.cls[0].item())
        conf    = float(box.conf[0].item())
        xyxyn   = box.xyxyn[0].tolist()   # normalised [x1, y1, x2,
y2]
        detections.append({
            "class"     : class_names[cls_id],
            "confidence": round(conf, 4),
            "bbox"      : xyxyn,
        })
    return detections


# ── Helper: draw detections on frame
# ──────────────────────────────────────────
COLOUR_MAP = {
    "crack"            : (0,   0,  255),
    "rust"             : (0, 140,  255),
    "corrosion"        : (0, 165,  255),
    "missing_bolt"     : (0, 255,  255),
    "paint_damage"     : (0, 200,  100),
    "structural_bend"  : (0,   0,  180),
    "hotspot_critical" : (0,   0,  220),
    "hotspot_moderate" : (0,  80,  255),
    "hotspot_minor"    : (30,165,  255),
    "default"          : (180,180, 180),
}

def draw_detections(frame, fused_records, top_n=10):
    h, w = frame.shape[:2]
    for rec in fused_records[:top_n]:
        if not rec["bbox"]:
            continue
        x1, y1, x2, y2 = rec["bbox"]
        px1, py1 = int(x1 * w), int(y1 * h)
        px2, py2 = int(x2 * w), int(y2 * h)
        color = COLOUR_MAP.get(rec["class"], COLOUR_MAP["default"])
        cv2.rectangle(frame, (px1, py1), (px2, py2), color, 2)
        label = f"{rec['class']} {rec['fused_score']:.2f}"
        cv2.putText(frame, label, (px1, max(py1 - 6, 10)),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.5, color, 1,
cv2.LINE_AA)
    return frame

print("Helpers defined.")
```

```
Helpers defined.

# ── Main Inference Replay Loop ──────────────────────────────────────

def run_replay(rgb_weights, thermal_weights, rgb_class_names,
thermal_class_names):
    """
    Run the full multi-modal fusion pipeline on INPUT_SOURCE.
    Works for both video files and single images.
    """
    if not Path(rgb_weights).exists():
        print("RGB weights not found. Train the model first.")
        return

    rgb_infer_model = YOLO(str(rgb_weights))
    th_infer_model  = YOLO(str(thermal_weights)) if (
        thermal_weights and Path(thermal_weights).exists()
    ) else None

    telem_lookup = load_telemetry_lookup(TELEMETRY_CSV)

    # Determine if input is image or video
    src = INPUT_SOURCE
    is_image = any(src.lower().endswith(ext) for ext in (".jpg",
".jpeg", ".png", ".bmp"))

    if is_image:
        frame = cv2.imread(src)
        frames = [frame]
    else:
        cap = cv2.VideoCapture(src)
        fps = cap.get(cv2.CAP_PROP_FPS) or 25
        total = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
        frames = []
        while True:
            ret, frm = cap.read()
            if not ret:
                break
            frames.append(frm)
        cap.release()
        print(f"Video loaded: {total} frames at {fps:.1f} fps")

    # Output video writer
    out_path = OUTPUT_DIR / ("output_replay.mp4" if not is_image else
"output_result.jpg")
    if not is_image:
        h, w = frames[0].shape[:2]
        writer = cv2.VideoWriter(
            str(out_path),
```

```
            cv2.VideoWriter_fourcc(*"mp4v"),
            fps, (w, h)
        )

    all_records = []

    for frame_id, frame in enumerate(tqdm(frames, desc="Processing
frames")):
        # RGB detection
        rgb_res    = rgb_infer_model(frame, conf=CONF_THRESHOLD,
                                     iou=IOU_THRESHOLD, verbose=False)
[0]
        rgb_dets   = parse_yolo_results(rgb_res, rgb_class_names)

        # Thermal detection (same frame — real system would use paired
stream)
        if th_infer_model is not None:
            th_res  = th_infer_model(frame, conf=CONF_THRESHOLD,
                                     iou=IOU_THRESHOLD, verbose=False)
[0]
            th_dets = parse_yolo_results(th_res, thermal_class_names)
        else:
            th_dets = []

        # Telemetry lookup
        if telem_lookup and frame_id in telem_lookup:
            telem_row = telem_lookup[frame_id]
        else:
            telem_row = DEFAULT_TELEM_ROW

        # Fuse
        fused = fuse_frame(rgb_dets, th_dets, telem_row)

        # Annotate frame
        annotated = draw_detections(frame.copy(), fused)

        if is_image:
            cv2.imwrite(str(out_path), annotated)
        else:
            writer.write(annotated)

        # Collect records
        for rec in fused:
            rec["frame_id"] = frame_id
            all_records.append(rec)

    if not is_image:
        writer.release()

    # Export report
```

```
    report_path = OUTPUT_DIR / "anomaly_report.csv"
    pd.DataFrame(all_records).to_csv(report_path, index=False)

    print(f"\nDone.")
    print(f"  Annotated output : {out_path}")
    print(f"  Anomaly report   : {report_path}")
    print(f"  Total anomaly detections: {len(all_records)}")
    return all_records


# ── Run replay ──────────────────────────────────────────────────────────
if Path(INPUT_SOURCE).exists():
    all_records = run_replay(
        rgb_weights      = str(RGB_BEST_WEIGHTS),
        thermal_weights  = str(THERMAL_BEST_WEIGHTS),
        rgb_class_names  = RGB_CLASSES,
        thermal_class_names = THERMAL_CLASSES,
    )
else:
    print(f"Input file '{INPUT_SOURCE}' not found.")
    print("Set INPUT_SOURCE to your video or image path and re-run
this cell.")
```

```
Processing frames: 100%|██████████| 1/1 [00:00<00:00,  1.28it/s]


Done.
  Annotated output : replay_output/output_result.jpg
  Anomaly report   : replay_output/anomaly_report.csv
  Total anomaly detections: 18
```

---

10 — Result Visualisation & Reporting

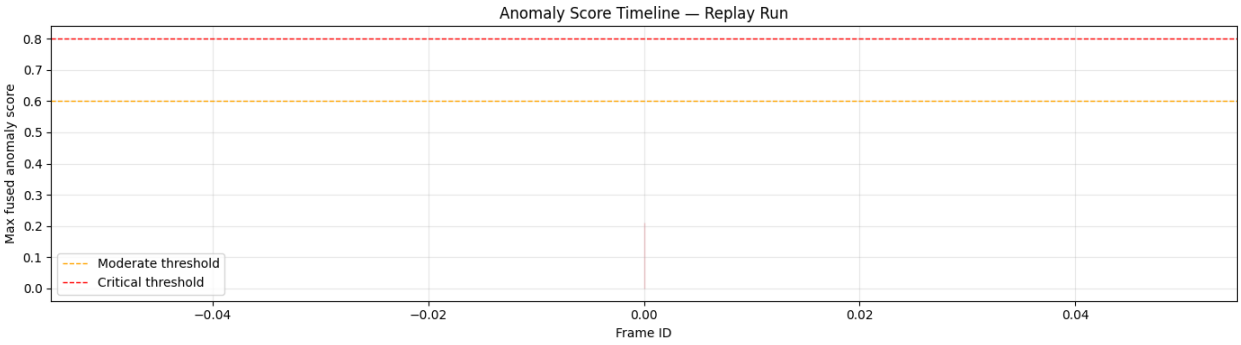
10.1 Anomaly Report Summary

```
# ── Load report (generated by run_replay) ──────────────────────────────
report_path = OUTPUT_DIR / "anomaly_report.csv"

if report_path.exists():
    report_df = pd.read_csv(report_path)
    print(f"Total records   : {len(report_df)}")
    print(f"Frames covered  : {report_df['frame_id'].nunique()}")
    print(f"Classes detected: {report_df['class'].nunique()}")
    print()
```

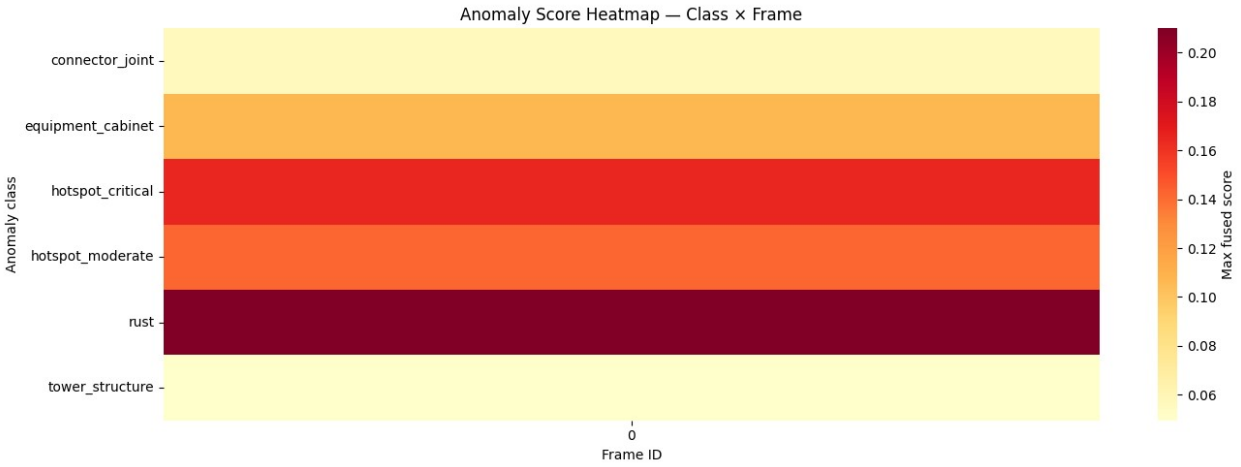
```
    print(report_df.groupby(["source", "class"])["fused_score"]
          .agg(["count", "mean", "max"])
          .round(3)
          .sort_values("mean", ascending=False)
          .to_string())
else:
    print("Report not generated yet — run replay first.")
```

```
Total records   : 18
Frames covered  : 1
Classes detected: 6

                           count   mean    max
source  class
rgb     rust                   2  0.199  0.210
thermal hotspot_critical       4  0.139  0.164
        hotspot_moderate       3  0.119  0.142
        equipment_cabinet      3  0.099  0.107
        connector_joint        5  0.050  0.056
        tower_structure        1  0.050  0.050
```

```
# ── Anomaly Score Timeline ──────────────────────────────────────
if report_path.exists():
    # Maximum fused_score per frame
    timeline = (report_df
                .groupby("frame_id")["fused_score"]
                .max()
                .reset_index())

    fig, ax = plt.subplots(figsize=(14, 4))
    ax.fill_between(timeline["frame_id"], timeline["fused_score"],
                    alpha=0.3, color="#e63946")
    ax.plot(timeline["frame_id"], timeline["fused_score"],
            color="#e63946", lw=1.5)
    ax.axhline(0.6, color="orange", lw=1, linestyle="--",
label="Moderate threshold")
    ax.axhline(0.8, color="red",    lw=1, linestyle="--",
label="Critical threshold")
    ax.set_xlabel("Frame ID")
    ax.set_ylabel("Max fused anomaly score")
    ax.set_title("Anomaly Score Timeline — Replay Run")
    ax.legend()
    ax.grid(alpha=0.3)
    plt.tight_layout()
    plt.savefig("anomaly_timeline.png", dpi=150)
    plt.show()
else:
    print("No report found.")
```




```
# ── Anomaly Class Frequency Heatmap across Frames ──────────────────────
if report_path.exists():
    pivot = (report_df
             .groupby(["frame_id", "class"])["fused_score"]
             .max()
             .unstack(fill_value=0))

    fig, ax = plt.subplots(figsize=(14, 5))
    sns.heatmap(pivot.T, ax=ax, cmap="YlOrRd", cbar_kws={"label": "Max
fused score"},
                linewidths=0, yticklabels=True)
    ax.set_title("Anomaly Score Heatmap — Class × Frame")
    ax.set_xlabel("Frame ID")
    ax.set_ylabel("Anomaly class")
    plt.tight_layout()
    plt.savefig("anomaly_heatmap.png", dpi=150)
    plt.show()
else:
    print("No report found.")
```




```
# ── Top 10 Highest-Severity Events ─────────────────────────────────────
```

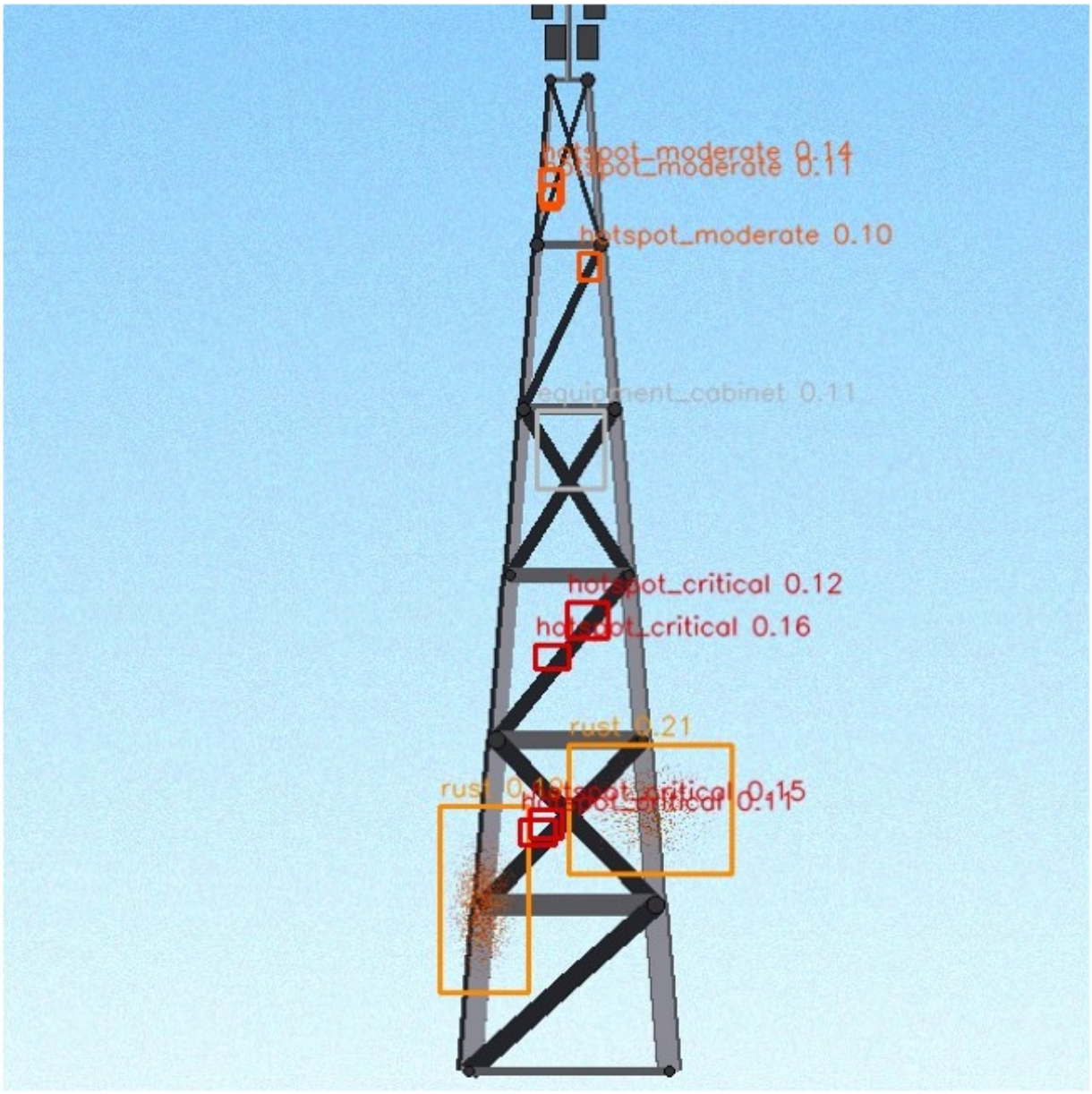
```
if report_path.exists():
    top_events = (report_df
                  .sort_values("fused_score", ascending=False)
                  .drop_duplicates(subset=["class", "frame_id"])
                  .head(10)
                  [["frame_id", "source", "class", "confidence",
"fused_score", "severity_weight", "group"]])

    print("=" * 80)
    print("TOP 10 ANOMALY EVENTS (sorted by fused score)")
    print("=" * 80)
    print(top_events.to_string(index=False))
else:
    print("No report found.")
```

```
================================================================================
TOP 10 ANOMALY EVENTS (sorted by fused score)
================================================================================
 frame_id  source             class  confidence  fused_score
severity_weight           group
        0     rgb              rust      0.8477       0.2098
0.55         surface
        0 thermal  hotspot_critical      0.4323       0.1643
0.95   thermal_fault
        0 thermal  hotspot_moderate      0.4737       0.1421
0.75 thermal_anomaly
        0 thermal equipment_cabinet      0.8895       0.1067
0.30         unknown
        0 thermal   connector_joint      0.4655       0.0559
0.30       connector
        0 thermal   tower_structure      0.4135       0.0496
0.30         unknown
```

```
# ── Display first annotated output frame ──────────────────────────
result_img_path = OUTPUT_DIR / "output_result.jpg"
result_vid_path = OUTPUT_DIR / "output_replay.mp4"

if result_img_path.exists():
    img = Image.open(result_img_path)
    plt.figure(figsize=(12, 7))
    plt.imshow(img)
    plt.axis("off")
    plt.title("Annotated Output — Fused Anomaly Detections",
fontsize=13)
    plt.tight_layout()
    plt.show()
elif result_vid_path.exists():
```

```
    # Extract first frame of video output for display
    cap = cv2.VideoCapture(str(result_vid_path))
    ret, frame = cap.read()
    cap.release()
    if ret:
        frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        plt.figure(figsize=(12, 7))
        plt.imshow(frame_rgb)
        plt.axis("off")
        plt.title("First Annotated Frame — Fused Anomaly Detections",
fontsize=13)
        plt.tight_layout()
        plt.show()
else:
    print("Output not found — run replay first.")
```

Annotated Output — Fused Anomaly Detections



Summary

What Was Built

Component	Details
RGB detector	YOLOv8s fine-tuned on 6 structural defect classes
Thermal detector	YOLOv8s fine-tuned on 8 thermal anomaly classes

Component	Details
Fusion layer	Weighted late fusion with semantic group agreement bonus
Telemetry integration	GPS reliability weighting, anomaly class cross-referencing
Replay pipeline	Frame-level inference on any video or image file
Reporting	CSV anomaly log, timeline chart, heatmap, top-event table

Fusion Weight Summary

$$S_{fused} = 0.45 \cdot c_{rgb} \cdot w_{rgb} + 0.40 \cdot c_{th} \cdot w_{th} + 0.15 \cdot c_{telem} \cdot w_{telem}$$

When two or more modalities agree on the same semantic anomaly group, the result is multiplied by a bonus factor of **1.15** to reward cross-modal consensus.

Telemetry confidence is pre-multiplied by a GPS reliability factor:

$$r = max\left(0, 1 - \frac{\text{HDOP} - 1}{5}\right)$$

Limitations and Future Work

- **Temporal fusion**: The current pipeline treats each frame independently. A recurrent module (e.g. ConvLSTM) could accumulate evidence across frames to reduce false positives from momentary noise.
- **True thermal pairing**: In this replay, the RGB model is applied to the thermal-like frame as a proxy. A real system would synchronise two cameras by timestamp.
- **Spatial GPS tagging**: Detected bounding boxes should be projected into GPS coordinates using the camera intrinsics, gimbal angle, and drone altitude for a fully geo-referenced defect map.
- **Active learning**: Low-confidence detections in the field should be flagged for human review and added to the training set iteratively.

Pipeline developed for drone-based 5G/LTE cell tower structural health monitoring. Datasets are synthetic drone-view simulations. All training and inference runs locally — no cloud dependency.